C# DSA

Data Structures & Algorithms

From Big-O to Dynamic Programming, with C# code, diagrams & patterns

- Complexity analysis & problem-solving framework
- Arrays, Strings, Hashing, Linked Lists, Stacks, Queues
- Trees, Heaps, Graphs, Tries & Union-Find
- Sorting, Searching, Recursion, Greedy & Dynamic Programming

Prepared by Kaif Shaikh • Coding Interview Edition

Table of Contents

1. Introduction to DSA 5

1.1 How to Use This Guide 5

1.2 What are Data Structures and Algorithms? 5

1.3 A Problem-Solving Framework (UMPIRE) 6

1.4 Constraints Tell You the Expected Complexity 6

1.5 C# Toolkit for DSA 7

2. Complexity Analysis (Big-O) 9

2.1 Why Analyse Complexity? 9

2.2 Common Complexity Classes 10

2.3 Rules for Calculating Big-O 10

2.4 Space Complexity 11

2.5 Best, Average, Worst and Amortised 11

2.6 Complexity of Recursive Algorithms 11

2.7 Complexity of C# Collection Operations 12

3. Arrays and Strings 13

3.1 Arrays 13

3.2 Prefix Sum 14

3.3 Two Pointers 14

3.4 Sliding Window 16

3.5 Kadane's Algorithm (Maximum Subarray) 18

3.6 Strings in C# 18

3.7 Matrix Patterns 20

4. Hashing 21

4.1 How a Hash Table Works 21

4.2 Core Patterns 22

4.3 When Hashing Is Not the Answer 23

5. Recursion and Backtracking 25

5.1 Recursion Basics 25

5.2 Recursion Tree and Memoization 26

5.3 Backtracking 26

6. Searching and Binary Search 29

6.1 Linear Search 29

6.2 Binary Search 29

6.3 Lower Bound and Upper Bound 30

6.4 Search in Rotated Sorted Array 31

6.5 Binary Search on the Answer 31

6.6 Searching a 2-D Matrix ... 32
7. Sorting Algorithms ... 33
7.1 Overview ... 33
7.2 Bubble, Selection and Insertion Sort ... 34
7.3 Merge Sort (Divide and Conquer) ... 35
7.4 Quick Sort ... 35
7.5 Heap Sort and Counting Sort ... 36
7.6 Dutch National Flag (Sort 0s, 1s, 2s) ... 37
7.7 Custom Sorting in C# ... 37
8. Linked Lists ... 38
8.1 Structure ... 38
8.2 Basic Operations ... 39
8.3 Reverse a Linked List ... 39
8.4 Fast and Slow Pointers (Floyd) ... 40
8.5 Classic Problems ... 41
8.6 LRU Cache (Dictionary + Doubly Linked List) ... 42
9. Stacks and Queues ... 43
9.1 Concepts ... 43
9.2 Stack Problems ... 44
9.3 Monotonic Stack ... 45
9.4 Queue Problems and Deque ... 47
10. Trees and Binary Search Trees ... 48
10.1 Terminology ... 48
10.2 Traversals ... 49
10.3 Classic Recursive Tree Problems ... 51
10.4 Binary Search Tree Operations ... 52
10.5 Build Trees and Serialize ... 53
11. Heaps and Priority Queues ... 54
11.1 What is a Heap? ... 54
11.2 PriorityQueue in C# (.NET 6+) ... 55
11.3 Implementing a Min-Heap ... 56
11.4 Top-K Pattern ... 56
11.5 Merge K Sorted Lists ... 57
11.6 Median of a Data Stream (Two Heaps) ... 58
12. Graphs ... 59
12.1 Terminology and Representation ... 59
12.2 Breadth-First Search (BFS) ... 60
12.3 Depth-First Search (DFS) ... 60
12.4 Grid Problems (Implicit Graphs) ... 61
12.5 Cycle Detection ... 63

12.6 Topological Sort (Kahn's Algorithm) ... 63
12.7 Shortest Paths ... 64
12.8 Minimum Spanning Tree (Kruskal) ... 65
13. Tries, Union-Find and Range Query Structures ... 66
13.1 Trie (Prefix Tree) ... 66
13.2 Union-Find (Disjoint Set Union) ... 67
13.3 Fenwick Tree (Binary Indexed Tree) ... 68
13.4 Segment Tree ... 68
14. Greedy Algorithms and Intervals ... 70
14.1 The Greedy Idea ... 70
14.2 Classic Greedy Problems ... 71
14.3 Interval Problems ... 72
15. Dynamic Programming ... 73
15.1 What is DP? ... 73
15.2 The DP Recipe ... 73
15.3 1-D DP ... 74
15.4 2-D DP ... 76
15.5 Other Common DP Families ... 78
16. Bit Manipulation and Math ... 80
16.1 Bitwise Operators ... 80
16.2 Essential Tricks ... 81
16.3 Math for Coding Interviews ... 82
17. Patterns, Interview Strategy and Revision ... 83
17.1 Problem-to-Pattern Cheat Sheet ... 83
17.2 Edge Case Checklist ... 84
17.3 Common C# Pitfalls in Coding Rounds ... 84
17.4 Must-Practise Problem List (by Topic) ... 85
17.5 Quick Quiz: Name the Pattern and Complexity ... 86
17.6 Study Roadmap ... 86

1. Introduction to DSA

1.1 How to Use This Guide

> ✎ **Definition**
>
> A study guide that explains each data structure and algorithm in the same order: the idea, a diagram, C# code with complexity, then tricky points and interview tips.

This guide teaches Data Structures and Algorithms (DSA) using **C#**, from complexity analysis to dynamic programming. Each topic follows the pattern: **intuition (often an analogy), then a diagram, then clean C# code with complexity, then tricky points**. Coloured boxes help you scan:

> ☼ **Simple Analogy**
>
> A real-world comparison that makes the idea stick.

> ⚡ **Tricky Point**
>
> Tricky points: edge cases and traps that break solutions in interviews.

> ★ **Interview Tip**
>
> Interview tip: how to explain the approach in one or two crisp lines.

> ⚠ **Watch Out**
>
> Watch out: common bugs (off-by-one, overflow, null references).

1.2 What are Data Structures and Algorithms?

> ✎ **Definition**
>
> A **data structure** is a way of organising data in memory so it can be accessed and changed efficiently. An **algorithm** is a finite, step-by-step procedure that solves a problem.

A **data structure** is a way of organising data in memory so it can be used efficiently (array, linked list, hash table, tree, graph). An **algorithm** is a finite, step-by-step procedure to solve a problem (sorting, searching, shortest path).

> ☼ **Simple Analogy**
>
> A library: the **data structure** is how books are arranged (by genre, alphabetically, with an index card catalogue). The **algorithm** is the procedure a librarian follows to find a book. A good arrangement makes the procedure fast.

Category	Data structures
Linear	Array, List, Linked List, Stack, Queue, Deque
Hash-based	Hash table (Dictionary), Hash set
Hierarchical	Tree, Binary Search Tree, Heap, Trie
Network	Graph (directed/undirected, weighted)
Specialised	Union-Find, Segment Tree, Fenwick Tree, LRU cache

Why DSA matters (beyond interviews)

In payments and reconciliation systems, choosing a `Dictionary` instead of nested loops turns an O(n x m) matching of 1 million switch records against 1 million bank records (10^12 comparisons, impossible) into O(n + m) (2 million operations, under a second). DSA is about writing code that still works when data grows.

1.3 A Problem-Solving Framework (UMPIRE)

> **✎ Definition**
>
> **UMPIRE** is a step-by-step method for solving interview problems: **U**nderstand, **M**atch to a known pattern, **P**lan, **I**mplement, **R**eview, **E**valuate complexity.

Use this routine for every problem, especially in interviews:

- **U - Understand:** restate the problem; ask about input size, value ranges, negatives, duplicates, empty input, sorted or not.
- **M - Match:** which pattern fits? (two pointers, sliding window, hashing, BFS, DP...). See the pattern table in Chapter 17.
- **P - Plan:** explain a brute-force solution first, state its complexity, then improve it.
- **I - Implement:** write clean code with good names.
- **R - Review:** dry-run with a small example; check edge cases.
- **E - Evaluate:** state final time and space complexity and possible trade-offs.

> **★ Interview Tip**
>
> Interviewers grade **communication** as much as code. Think aloud, start with brute force, and justify each optimisation with complexity.

1.4 Constraints Tell You the Expected Complexity

> **✎ Definition**
>
> The input size limits in a problem show which time complexity is acceptable. For example, n up to 10^5 needs about O(n log n) or better, while n up to 20 allows O(2^n).

For a typical 1-2 second limit (~10^8 simple operations):

Input size n	Acceptable complexity	Typical approach
n <= 10-12	O(n!), O(2^n * n)	Permutations, backtracking
n <= 20-25	O(2^n)	Subsets, bitmask DP
n <= 500	O(n^3)	Floyd-Warshall, 3 nested loops
n <= 5,000	O(n^2)	Simple DP, double loop
n <= 10^6	O(n log n)	Sorting, heap, binary search
n <= 10^8	O(n)	Single pass, two pointers, hashing
n > 10^8	O(log n) or O(1)	Math, binary search on answer

1.5 C# Toolkit for DSA

✎ Definition

The C# types and APIs used most in DSA problems: arrays, `List`, `Dictionary`, `HashSet`, `Stack`, `Queue`, `PriorityQueue`, `StringBuilder`, sorting and LINQ helpers.

```
using System.Text;
using System.Collections.Generic;

int[] arr = new int[n];                    // fixed-size array, zero-initialised
int[,] grid = new int[rows, cols];         // 2-D matrix
int[][] jag = new int[n][];                // jagged (rows of different lengths)
var list = new List<int>();                // dynamic array
var map = new Dictionary<int, int>();      // hash map
var set = new HashSet<int>();              // hash set
var st = new Stack<int>();                 // LIFO
```

```
var q = new Queue<int>();                  // FIFO
var pq = new PriorityQueue<int, int>();    // min-heap (.NET 6+)
var sorted = new SortedSet<int>();         // balanced BST (red-black tree)
var sb = new StringBuilder();              // efficient string building

Array.Sort(arr);                           // in-place introsort, O(n log n)
Array.Sort(arr, (a, b) => b.CompareTo(a)); // descending
Array.Fill(arr, -1);
int idx = Array.BinarySearch(arr, 5);      // negative (~insertionPoint) if absent
int max = arr.Max(); int sum = arr.Sum();  // LINQ (convenient, a bit slower)
(a, b) = (b, a);                           // swap with tuples
int INF = int.MaxValue; long big = long.MaxValue;
```

⚡ Tricky Point

Integer overflow is silent in C# by default. `int.MaxValue + 1` becomes `int.MinValue`. Use `long` for sums/products of large values, compute midpoints as `lo + (hi - lo) / 2`, and be careful with `INF + x` when `INF = int.MaxValue`.

⚡ Tricky Point

Prefer `a.CompareTo(b)` over `a - b` in comparators: `int.MinValue - 1` overflows and produces a wrong sign.

Fast input/output for competitive programming

```
using var reader = new StreamReader(Console.OpenStandardInput(), bufferSize: 1 << 16);
using var writer = new StreamWriter(Console.OpenStandardOutput(), bufferSize: 1 << 16);
int n = int.Parse(reader.ReadLine()!);
int[] nums = reader.ReadLine()!.Split(' ', StringSplitOptions.RemoveEmptyEntries).Select(int.Parse).ToArray();
writer.WriteLine(nums.Sum());
```

2. Complexity Analysis (Big-O)

2.1 Why Analyse Complexity?

> ✎ **Definition**
>
> **Complexity analysis** estimates how an algorithm's running time and memory grow as the input grows. It lets you compare solutions without running them.

Measuring time with a stopwatch depends on hardware, language and input. **Asymptotic analysis** describes how the number of operations **grows** as the input size *n* grows - independent of the machine.

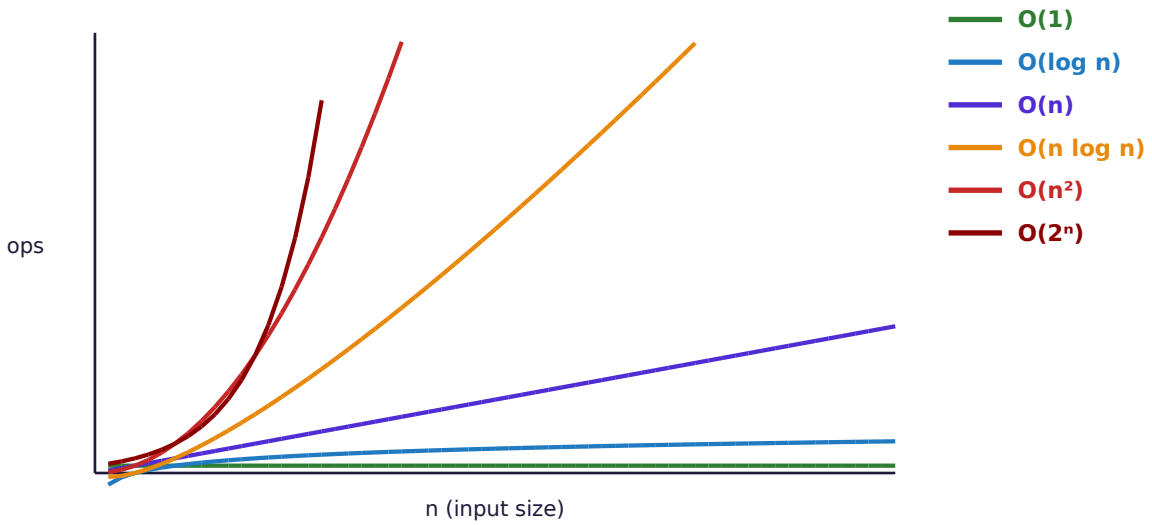


Figure 2.1 - Growth of common complexity classes

Notation	Meaning	Analogy
O (Big-O)	Upper bound: grows no faster than	"At most 30 minutes"
Omega	Lower bound: grows at least as fast as	"At least 10 minutes"
Theta	Tight bound: both upper and lower	"Exactly about 20 minutes"

In interviews, "Big-O" usually means the **worst-case tight bound**.

2.2 Common Complexity Classes

> **✎ Definition**
>
> The standard growth rates from fastest to slowest: O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(2^n)$, O(n!).

Big-O	Name	Example	n = 1,000,000
O(1)	Constant	Array index, Dictionary lookup (avg), stack push	1
O(log n)	Logarithmic	Binary search, balanced BST, heap push	~20
O(n)	Linear	Single loop, linear search	10^6
O(n log n)	Linearithmic	Merge sort, heap sort, Array.Sort	~2 x 10^7
O(n^2)	Quadratic	Nested loops, bubble sort	10^12 (too slow)
O(n^3)	Cubic	Triple nested loops	10^18

Big-O	Name	Example	n = 1,000,000
O(2^n)	Exponential	All subsets, naive Fibonacci	impossible
O(n!)	Factorial	All permutations	impossible

2.3 Rules for Calculating Big-O

> **✎ Definition**
>
> Rules for working out Big-O: drop constants, keep only the dominant term, **add** the costs of sequential steps, **multiply** the costs of nested loops, and use different variables for different inputs.

- **Drop constants:** O(2n + 5) = O(n).
- **Drop lower-order terms:** O(n^2 + n) = O(n^2).
- **Sequential steps add:** loop A then loop B = O(a + b).
- **Nested steps multiply:** loop over A containing loop over B = O(a * b).
- **Different inputs, different variables:** two arrays of sizes n and m give O(n + m) or O(n * m), not O(n^2).

```
void Example(int[] a, int[] b)
{
    for (int i = 0; i < a.Length; i++) Console.Write(a[i]);       // O(n)
    for (int i = 0; i < a.Length; i++)                              // O(n * m)
        for (int j = 0; j < b.Length; j++) Console.Write(a[i] + b[j]);
}   // total: O(n + n*m) = O(n*m)

for (int i = 1; i < n; i *= 2) { }          // O(log n): i doubles each step
for (int i = 0; i < n; i++)                  // O(n log n)
    for (int j = 1; j < n; j *= 2) { }
for (int i = 0; i < n; i++)                  // O(n^2): 0+1+2+...+(n-1) = n(n-1)/2
    for (int j = 0; j < i; j++) { }
```

> ⚡ **Tricky Point**
>
> **Hidden costs in C#:** `list.Contains(x)`, `list.Remove(x)`, `list.Insert(0, x)`, `list.IndexOf(x)`, `string.Substring`, `s1 + s2` and LINQ `ToList()` are all **O(n)**. A loop that calls one of them is O(n^2), not O(n).

2.4 Space Complexity

> ✎ **Definition**
>
> **Space complexity** is the extra memory an algorithm uses as a function of input size, including auxiliary data structures and the recursion call stack.

Extra memory used by the algorithm (auxiliary space), not counting the input. Recursion uses **stack space** proportional to the maximum recursion depth.

```
int SumIterative(int[] a) { int s = 0; foreach (var x in a) s += x; return s; }  // O(1) space
int SumRecursive(int[] a, int i) => i == a.Length ? 0 : a[i] + SumRecursive(a, i + 1);  // O(n) stack
```

2.5 Best, Average, Worst and Amortised

> ✎ **Definition**
>
> **Best**, **average** and **worst** case describe cost for the most favourable, typical and least favourable inputs. **Amortised** cost is the average per operation over a sequence of operations.

- **Quick sort:** best/average O(n log n), worst O(n^2) (bad pivots on sorted data).
- **Dictionary lookup:** average O(1), worst O(n) (all keys collide).
- **Amortised:** `List<T>.Add` is usually O(1), but occasionally O(n) when the internal array doubles. Averaged over many adds it is **amortised O(1)**.

2.6 Complexity of Recursive Algorithms

> ✎ **Definition**
>
> Finding the cost of recursion from the **recursion tree** (number of calls × work per call) or a recurrence relation, e.g. T(n) = 2T(n/2) + O(n) gives O(n log n).

Recurrence	Example	Result
T(n) = T(n/2) + O(1)	Binary search	O(log n)
T(n) = T(n-1) + O(1)	Linear recursion (factorial)	O(n)
T(n) = 2T(n/2) + O(n)	Merge sort	O(n log n)
T(n) = 2T(n-1) + O(1)	Naive Fibonacci, Tower of Hanoi	O(2^n)
T(n) = T(n-1) + O(n)	Selection sort recursively	O(n^2)

Shortcut: time of a recursive algorithm = (number of calls) x (work per call). For a recursion tree with branching factor *b* and depth *d*, calls are about b^d.

2.7 Complexity of C# Collection Operations

✎ Definition

The Big-O cost of common operations on .NET collections, such as `List.Add` amortised O(1), `Insert` O(n), `Dictionary` lookup O(1) average, and `SortedSet` O(log n).

Operation	Array / List<T>	LinkedList<T>	Dictionary / HashSet	SortedSet / SortedDictionary	PriorityQueue
Access by index	O(1)	O(n)	-	-	-
Search	O(n)	O(n)	O(1) avg	O(log n)	O(n)
Insert end / add	O(1) amortised	O(1)	O(1) avg	O(log n)	O(log n)
Insert at front/middle	O(n)	O(1) with node	-	-	-
Delete	O(n)	O(1) with node	O(1) avg	O(log n)	O(log n) min
Min / Max	O(n)	O(n)	O(n)	O(log n) (`Min`/`Max`)	O(1) peek

★ Interview Tip

When asked for complexity, always state **both time and space**, mention the **worst case** and explain **where** it comes from ("the sort dominates, O(n log n); the hash set uses O(n) extra space").

3. Arrays and Strings

3.1 Arrays

> ✎ **Definition**
>
> An **array** stores elements of one type in **contiguous memory**, so access by index is O(1). Inserting or deleting in the middle costs O(n) because elements must shift.

An array stores elements of the same type in **contiguous memory**, so any element can be reached in O(1) by computing its address.

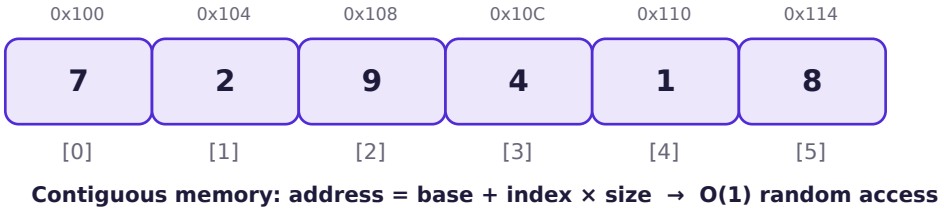


Figure 3.1 - An array in memory

Operation	Complexity
Access `a[i]`	O(1)
Update `a[i] = x`	O(1)
Search (unsorted)	O(n)
Search (sorted, binary search)	O(log n)
Insert/delete at end (List<T>)	O(1) amortised
Insert/delete at start/middle	O(n) - shift elements

Basic traversals

```
int[] a = { 7, 2, 9, 4, 1, 8 };
int max = int.MinValue, min = int.MaxValue; long sum = 0;
foreach (int x in a) { max = Math.Max(max, x); min = Math.Min(min, x); sum += x; }

// Reverse in place - O(n) time, O(1) space
void Reverse(int[] arr, int l, int r)
{
    while (l < r) { (arr[l], arr[r]) = (arr[r], arr[l]); l++; r--; }
}
// Rotate right by k: reverse all, reverse first k, reverse rest
void Rotate(int[] nums, int k)
{
    k %= nums.Length;                          // k may be larger than n!
    Reverse(nums, 0, nums.Length - 1);
    Reverse(nums, 0, k - 1);
    Reverse(nums, k, nums.Length - 1);
}
```

> ⚡ **Tricky Point**
>
> Rotating by `k` where `k >= n` - always do `k %= n`. Also handle `n == 0` before the modulo (division by zero).

2-D arrays (matrices)

```
int[,] m = new int[3, 4];
for (int r = 0; r < m.GetLength(0); r++)
    for (int c = 0; c < m.GetLength(1); c++)
        m[r, c] = r * 4 + c;

int[][] grid = { new[] { 1, 2 }, new[] { 3, 4 } };      // jagged - common on LeetCode
int[] dr = { -1, 1, 0, 0 }, dc = { 0, 0, -1, 1 };       // 4-direction moves
for (int d = 0; d < 4; d++)
{
    int nr = r + dr[d], nc = c + dc[d];
    if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) continue;   // bounds check first!
}
```

3.2 Prefix Sum

✎ Definition

A **prefix sum** array stores running totals (`prefix[i] = sum of first i elements`), so the sum of any subarray is answered in O(1) after O(n) preprocessing.

Pre-compute cumulative sums so that **any range sum is O(1)**.

```
// prefix[i] = sum of a[0..i-1]; prefix has n+1 elements
long[] BuildPrefix(int[] a)
{
    var p = new long[a.Length + 1];
    for (int i = 0; i < a.Length; i++) p[i + 1] = p[i] + a[i];
    return p;
}
long RangeSum(long[] p, int l, int r) => p[r + 1] - p[l];     // sum of a[l..r] inclusive
// a = {2, 4, 6, 8}: p = {0, 2, 6, 12, 20}; RangeSum(1, 2) = p[3] - p[1] = 12 - 2 = 10
```

☼ Simple Analogy

A running bank balance: to know how much you spent between the 5th and the 20th, subtract the balance on the 4th from the balance on the 20th - no need to re-add every transaction.

3.3 Two Pointers

✎ Definition

Two pointers uses two indices moving through the data (from both ends, or one fast and one slow) to solve pair, partition and in-place problems in O(n).

Two indices move through the array (usually from both ends, or one fast and one slow) to avoid a nested loop.



Figure 3.2 - Two pointers on a sorted array

```
// Pair with target sum in a SORTED array: O(n) time, O(1) space
int[] TwoSumSorted(int[] a, int target)
{
    int l = 0, r = a.Length - 1;
    while (l < r)
    {
        int s = a[l] + a[r];
        if (s == target) return new[] { l, r };
        if (s < target) l++; else r--;
    }
    return Array.Empty<int>();
}

// Remove duplicates from sorted array in place; returns new length
int RemoveDuplicates(int[] a)
{
    if (a.Length == 0) return 0;
    int write = 1;                                  // slow pointer
    for (int read = 1; read < a.Length; read++)     // fast pointer
        if (a[read] != a[read - 1]) a[write++] = a[read];
    return write;
}

// Container with most water
int MaxArea(int[] h)
{
    int l = 0, r = h.Length - 1, best = 0;
    while (l < r)
    {
        best = Math.Max(best, Math.Min(h[l], h[r]) * (r - l));
        if (h[l] < h[r]) l++; else r--;             // move the shorter wall
    }
    return best;
}

// Move zeroes to the end keeping order
void MoveZeroes(int[] a)
{
    int w = 0;
    for (int i = 0; i < a.Length; i++)
        if (a[i] != 0) { (a[w], a[i]) = (a[i], a[w]); w++; }
}
```

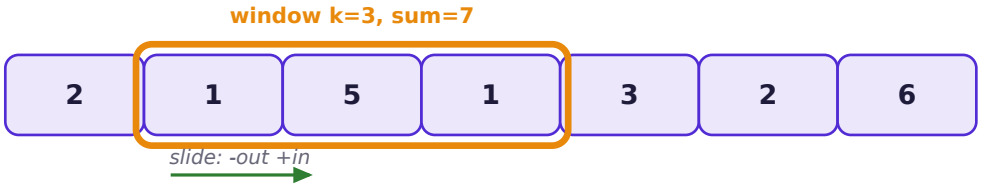
Three Sum (classic)

```
IList<IList<int>> ThreeSum(int[] nums)
{
    Array.Sort(nums);
    var res = new List<IList<int>>();
    for (int i = 0; i < nums.Length - 2; i++)
    {
        if (i > 0 && nums[i] == nums[i - 1]) continue;          // skip duplicate anchors
        int l = i + 1, r = nums.Length - 1;
        while (l < r)
        {
            int s = nums[i] + nums[l] + nums[r];
            if (s == 0)
            {
                res.Add(new List<int> { nums[i], nums[l], nums[r] });
                while (l < r && nums[l] == nums[l + 1]) l++;     // skip duplicates
                while (l < r && nums[r] == nums[r - 1]) r--;
                l++; r--;
            }
            else if (s < 0) l++; else r--;
        }
    }
    return res;   // O(n^2) time, O(1) extra (ignoring output and sort)
}
```

3.4 Sliding Window

> **✎ Definition**
>
> **Sliding window** keeps a range [left, right] that expands and shrinks over the array or string, solving contiguous subarray or substring problems in O(n) instead of $O(n^2)$.

Maintain a window `[left, right]` over a contiguous subarray/substring and slide it, updating the answer incrementally.



Add the new element, subtract the one that left: O(n) instead of O(n·k)

Figure 3.3 - Fixed-size sliding window

```
// Fixed size: max sum of any subarray of size k - O(n)
long MaxSumK(int[] a, int k)
{
    long window = 0;
    for (int i = 0; i < k; i++) window += a[i];
    long best = window;
    for (int i = k; i < a.Length; i++)
    {
        window += a[i] - a[i - k];              // add incoming, remove outgoing
        best = Math.Max(best, window);
    }
    return best;
}

// Variable size: longest substring without repeating characters - O(n)
int LengthOfLongestSubstring(string s)
{
    var lastSeen = new Dictionary<char, int>();
    int best = 0, left = 0;
    for (int right = 0; right < s.Length; right++)
    {
        if (lastSeen.TryGetValue(s[right], out int prev) && prev >= left)
            left = prev + 1;                        // shrink past the duplicate
        lastSeen[s[right]] = right;
        best = Math.Max(best, right - left + 1);
    }
    return best;
}

// Variable size: smallest subarray with sum >= target (positive numbers)
int MinSubArrayLen(int target, int[] nums)
{
    int left = 0, sum = 0, best = int.MaxValue;
    for (int right = 0; right < nums.Length; right++)
    {
        sum += nums[right];
        while (sum >= target)                       // shrink while valid
        {
            best = Math.Min(best, right - left + 1);
            sum -= nums[left++];
        }
    }
    return best == int.MaxValue ? 0 : best;
}
```

⚡ Tricky Point

The shrink-while-valid sliding window works only when expanding the window moves the sum in **one direction** (non-negative numbers). With negative numbers use **prefix sums + hash map** (Chapter 4) instead.

★ Interview Tip

Template for variable windows: expand `right` every step; while the window is invalid (or valid, for minimum problems) shrink `left`; update the answer at the right moment.

3.5 Kadane's Algorithm (Maximum Subarray)

✎ Definition

Kadane's algorithm finds the maximum-sum contiguous subarray in O(n) by tracking the best sum that ends at each position.

```
int MaxSubArray(int[] a)
{
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.Length; i++)
    {
        cur = Math.Max(a[i], cur + a[i]);     // extend or start fresh
        best = Math.Max(best, cur);
    }
    return best;          // O(n) time, O(1) space
}
// a = {-2,1,-3,4,-1,2,1,-5,4} -> 6 (subarray 4,-1,2,1)
```

⚡ Tricky Point

Initialise with `a[0]`, not 0: if every element is negative, the answer is the largest negative number, not 0.

3.6 Strings in C#

✎ Definition

String handling for DSA in C#: strings are **immutable**, so use `StringBuilder` or `char[]` for building and modifying, and know the common string, char and frequency-count techniques.

`string` is **immutable**: every concatenation creates a new string (O(n) copy). Use `StringBuilder` or a `char[]` for building/modifying.

```
string s = "Hello";
char c = s[1];                                  // 'e', O(1)
char[] arr = s.ToCharArray(); arr[0] = 'J'; string t = new string(arr);   // "Jello"
int idx = c - 'a';                              // char to 0-25 index (lowercase letters)
char back = (char)('a' + idx);
bool digit = char.IsDigit(c), letter = char.IsLetterOrDigit(c);
char lower = char.ToLower(c);
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.Append(i);  // O(total length) - not O(n^2)
```

Classic string problems

```
bool IsPalindrome(string s)                          // ignore non-alphanumerics, case
{
    int l = 0, r = s.Length - 1;
    while (l < r)
    {
        while (l < r && !char.IsLetterOrDigit(s[l])) l++;
        while (l < r && !char.IsLetterOrDigit(s[r])) r--;
        if (char.ToLower(s[l]) != char.ToLower(s[r])) return false;
        l++; r--;
    }
    return true;
}

bool IsAnagram(string a, string b)                   // O(n) with counting array
{
    if (a.Length != b.Length) return false;
    var count = new int[26];
    for (int i = 0; i < a.Length; i++) { count[a[i] - 'a']++; count[b[i] - 'a']--; }
    return count.All(x => x == 0);
}
string LongestPalindrome(string s)                   // expand around center, O(n^2)
{
    int start = 0, len = 0;
    for (int i = 0; i < s.Length; i++)
        foreach (var (l0, r0) in new[] { (i, i), (i, i + 1) })   // odd and even centres
        {
            int l = l0, r = r0;
            while (l >= 0 && r < s.Length && s[l] == s[r]) { l--; r++; }
            if (r - l - 1 > len) { len = r - l - 1; start = l + 1; }
        }
    return s.Substring(start, len);
}

string ReverseWords(string s) =>
    string.Join(' ', Enumerable.Reverse(s.Split(' ', StringSplitOptions.RemoveEmptyEntries)));
```

⚡ Tricky Point

Calling `.Reverse()` on an **array** variable is risky in newer C#: with C# 14 first-class spans it can bind to the in-place `Span` extension (returns void) instead of LINQ. Use `Array.Reverse(arr)` to reverse in place, or `Enumerable.Reverse(arr)` for a reversed sequence.

⚡ Tricky Point

Comparing strings: `==` compares content in C# (unlike Java). Use `string.CompareOrdinal` or `StringComparer.Ordinal` for sorting by raw character codes; culture-sensitive comparison can give unexpected orders.

3.7 Matrix Patterns

✎ Definition

Techniques for 2-D grids: traversing rows and columns, directions arrays, spiral order, rotation, transposition, and in-place marking.

```
// Transpose + reverse rows = rotate 90 degrees clockwise (n x n), in place
void Rotate(int[][] m)
{
    int n = m.Length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) (m[i][j], m[j][i]) = (m[j][i], m[i][j]);
    foreach (var row in m) Array.Reverse(row);
}

// Spiral order
```

```
IList<int> Spiral(int[][] m)
{
    var res = new List<int>();
    int top = 0, bottom = m.Length - 1, left = 0, right = m[0].Length - 1;
    while (top <= bottom && left <= right)
    {
        for (int c = left; c <= right; c++) res.Add(m[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) res.Add(m[r][right]);
        right--;
        if (top <= bottom) { for (int c = right; c >= left; c--) res.Add(m[bottom][c]); bottom--; }
        if (left <= right) { for (int r = bottom; r >= top; r--) res.Add(m[r][left]); left++; }
    }
    return res;
}
```

4. Hashing

4.1 How a Hash Table Works

> ✎ **Definition**
>
> A **hash table** maps keys to array buckets with a **hash function**, so insert, delete and lookup take O(1) on average. Collisions are handled by chaining or probing.

A hash table maps a key to a bucket using a **hash function**, giving **average O(1)** insert, delete and lookup. In C#: `Dictionary<TKey, TValue>` and `HashSet<T>`.

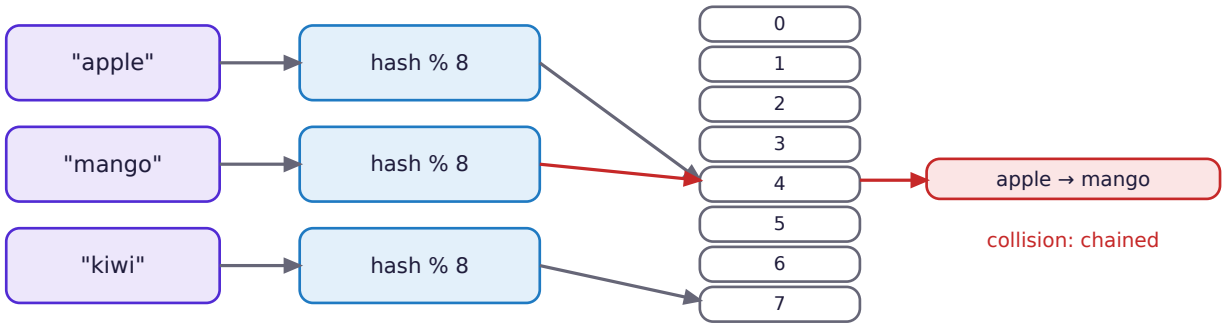


Figure 4.1 - Keys hashed into buckets; collisions are chained

- Hash function: `key.GetHashCode()`, then reduced to a bucket index.
- **Collision:** two keys in the same bucket. .NET chains entries in a bucket and compares with `Equals`.
- **Load factor:** when the table fills, it resizes (about doubles) and rehashes - amortised O(1).
- Worst case O(n) if many keys collide (bad hash function or adversarial input).

> ☼ **Simple Analogy**
>
> A hash table is like a coat-check counter. You hand over a coat (value) and get a numbered token (hash). To retrieve it, the attendant goes straight to hook number N instead of searching every hook.

> ⚡ **Tricky Point**
>
> Custom classes as keys need proper `Equals` and `GetHashCode` overrides (or use `record`/tuples, which have value equality). Never mutate a key while it is in the dictionary.

4.2 Core Patterns

> **✎ Definition**
>
> The standard hashing patterns: **frequency counting**, **complement lookup** (Two Sum), **seen-set** for duplicates, **grouping by a key** (anagrams), and **prefix-sum + map**.

Pattern 1: Complement lookup (Two Sum)

```
int[] TwoSum(int[] nums, int target)
{
    var seen = new Dictionary<int, int>();       // value -> index
    for (int i = 0; i < nums.Length; i++)
    {
        if (seen.TryGetValue(target - nums[i], out int j)) return new[] { j, i };
        seen[nums[i]] = i;                        // add AFTER checking (don't pair with itself)
    }
    return Array.Empty<int>();                    // O(n) time, O(n) space
}
```

Pattern 2: Frequency counting

```
var freq = new Dictionary<int, int>();
foreach (int x in nums) freq[x] = freq.GetValueOrDefault(x) + 1;

// First unique character
int FirstUniqChar(string s)
{
    var cnt = new int[26];
    foreach (char c in s) cnt[c - 'a']++;
    for (int i = 0; i < s.Length; i++) if (cnt[s[i] - 'a'] == 1) return i;
    return -1;
}

// Majority element (Boyer-Moore voting) - O(1) space alternative
int Majority(int[] a)
{
    int cand = 0, count = 0;
    foreach (int x in a) { if (count == 0) cand = x; count += x == cand ? 1 : -1; }
    return cand;
}
```

Pattern 3: Grouping by a canonical key

```
IList<IList<string>> GroupAnagrams(string[] strs)
{
    var groups = new Dictionary<string, List<string>>();
    foreach (var s in strs)
    {
        var chars = s.ToCharArray(); Array.Sort(chars);
        string key = new string(chars);                    // "eat","tea","ate" -> "aet"
        if (!groups.TryGetValue(key, out var list)) groups[key] = list = new List<string>();
        list.Add(s);
    }
    return groups.Values.ToList<IList<string>>();          // O(n * k log k)
}
```

Pattern 4: Existence checks with HashSet

```
bool ContainsDuplicate(int[] nums)
{
    var set = new HashSet<int>();
    foreach (int x in nums) if (!set.Add(x)) return true;   // Add returns false if present
    return false;
}

int LongestConsecutive(int[] nums)                          // O(n)
{
    var set = new HashSet<int>(nums);
    int best = 0;
    foreach (int x in set)
    {
        if (set.Contains(x - 1)) continue;                  // only start at sequence beginnings
        int len = 1;
        while (set.Contains(x + len)) len++;
        best = Math.Max(best, len);
    }
    return best;
}
```

> ⚡ **Tricky Point**
>
> In `LongestConsecutive`, the "only start when x-1 is absent" check is what makes it O(n). Without it, each element walks its whole sequence: O(n^2).

Pattern 5: Prefix sum + hash map (subarray sum equals K)

Works with **negative numbers**, where sliding window fails.

```
int SubarraySum(int[] nums, int k)
{
    var count = new Dictionary<long, int> { [0] = 1 };    // empty prefix
    long prefix = 0; int res = 0;
    foreach (int x in nums)
    {
        prefix += x;
        res += count.GetValueOrDefault(prefix - k);       // earlier prefix that makes sum k
        count[prefix] = count.GetValueOrDefault(prefix) + 1;
    }
    return res;     // O(n)
}
```

> ★ **Interview Tip**
>
> Idea in one line: "sum(i..j) = prefix[j] - prefix[i-1] = k, so for each j I count how many earlier prefixes equal prefix[j] - k."

4.3 When Hashing Is Not the Answer

> ✎ **Definition**
>
> Cases where a hash table is the wrong tool: when you need order or range queries, when memory is tight, or when sorting or two pointers already give a simpler solution.

- You need **ordering** (min, max, range queries): use `SortedSet`/`SortedDictionary` (O(log n)) or sort first.
- Keys are small integers in a known range: a plain **array** of counts is faster than a dictionary.

- Memory is tight and the array can be sorted: sorting + two pointers uses O(1) extra space.

5. Recursion and Backtracking

5.1 Recursion Basics

> ✎ **Definition**
>
> **Recursion** is a function calling itself on a smaller sub-problem. It needs a **base case** to stop and a **recursive case** that moves towards it.

A recursive function **calls itself** on a smaller version of the problem. Every recursive function needs:

- a **base case** that stops the recursion, and
- a **recursive case** that moves towards the base case.

> ☼ **Simple Analogy**
>
> Russian nesting dolls: to count the dolls, open one doll and count the rest the same way. When you reach the smallest doll that doesn't open (base case), you stop and add up on the way back.

```
long Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);          // O(n) time, O(n) stack

long Power(long x, int n)                                              // fast exponentiation O(log n)
{
    if (n == 0) return 1;
    long half = Power(x, n / 2);
    return n % 2 == 0 ? half * half : half * half * x;
}

int SumDigits(int n) => n < 10 ? n : n % 10 + SumDigits(n / 10);
```

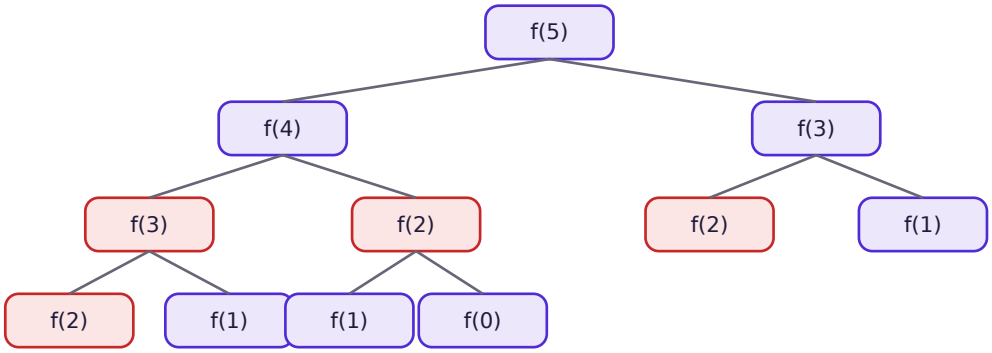
The call stack

Each call gets a **stack frame** (parameters, locals, return address). Frames are popped in reverse order when calls return. Too many frames cause a `StackOverflowException`, which **cannot be caught** in .NET and terminates the process.

> ⚠ **Watch Out**
>
> In C#, recursion depth around 10,000-100,000 (depending on frame size and the 1 MB default stack) can crash. For deep inputs (long linked lists, 10^5-node degenerate trees, large DFS on grids) prefer an **iterative** version with an explicit `Stack<T>`.

5.2 Recursion Tree and Memoization

> **✎ Definition**
>
> A **recursion tree** shows every call a recursive function makes. **Memoization** caches the result of each sub-problem so repeated calls are answered instantly.



Fibonacci: repeated sub-problems (red) → O(2ⁿ). Memoization makes it O(n).

Figure 5.1 - Naive Fibonacci recomputes the same sub-problems

```
int FibNaive(int n) => n < 2 ? n : FibNaive(n - 1) + FibNaive(n - 2);    // O(2^n)

long FibMemo(int n, Dictionary<int, long> memo)
{
    if (n < 2) return n;
    if (memo.TryGetValue(n, out var v)) return v;
    return memo[n] = FibMemo(n - 1, memo) + FibMemo(n - 2, memo);      // O(n)
}
```

Memoization (caching results of sub-problems) is the bridge to **Dynamic Programming** (Chapter 15).

5.3 Backtracking

> **✎ Definition**
>
> **Backtracking** builds a solution step by step, **choosing** an option, **exploring** further and **undoing** the choice to try others. Used for permutations, subsets and puzzles.

Backtracking builds a solution step by step, and **undoes** (backtracks) a choice when it can't lead to a valid solution. Template:

```
void Backtrack(State state, List<Choice> path, List<Result> results)
{
    if (IsComplete(state)) { results.Add(Snapshot(path)); return; }
    foreach (var choice in Choices(state))
    {
        if (!IsValid(choice)) continue;     // prune
        path.Add(choice);                   // choose
        Backtrack(Next(state, choice), path, results);   // explore
        path.RemoveAt(path.Count - 1);      // un-choose (backtrack)
    }
}
```

> ⚡ **Tricky Point**
>
> Always add a **copy** of the path (`new List<int>(path)`) to the results. Adding `path` itself stores a reference that you keep modifying - at the end every result is the same empty list.

Subsets (power set) - O(n * 2^n)

```
IList<IList<int>> Subsets(int[] nums)
{
    var res = new List<IList<int>>();
    void Dfs(int start, List<int> path)
    {
        res.Add(new List<int>(path));                // every node is a subset
        for (int i = start; i < nums.Length; i++)
        {
            path.Add(nums[i]);
            Dfs(i + 1, path);
            path.RemoveAt(path.Count - 1);
        }
    }
    Dfs(0, new List<int>());
    return res;
}
```

Permutations - O(n * n!)

```
IList<IList<int>> Permute(int[] nums)
{
    var res = new List<IList<int>>();
    var used = new bool[nums.Length];
    var path = new List<int>();
    void Dfs()
    {
        if (path.Count == nums.Length) { res.Add(new List<int>(path)); return; }
        for (int i = 0; i < nums.Length; i++)
        {
            if (used[i]) continue;
            used[i] = true; path.Add(nums[i]);
            Dfs();
            used[i] = false; path.RemoveAt(path.Count - 1);
        }
    }
    Dfs();
    return res;
}
```

Combination Sum (reuse allowed)

```
IList<IList<int>> CombinationSum(int[] candidates, int target)
{
    Array.Sort(candidates);
    var res = new List<IList<int>>();
    void Dfs(int start, int remain, List<int> path)
    {
        if (remain == 0) { res.Add(new List<int>(path)); return; }
        for (int i = start; i < candidates.Length && candidates[i] <= remain; i++)   // prune
        {
            path.Add(candidates[i]);
            Dfs(i, remain - candidates[i], path);        // i, not i+1: same number reusable
            path.RemoveAt(path.Count - 1);
        }
    }
    Dfs(0, target, new List<int>());
    return res;
}
```

⚡ Tricky Point

Handling **duplicates** in input (Subsets II, Permutations II, Combination Sum II): sort first, then skip `if (i > start && nums[i] == nums[i - 1]) continue;` so equal values at the same tree level are used only once.

N-Queens

```
IList<IList<string>> SolveNQueens(int n)
{
    var res = new List<IList<string>>();
    var cols = new bool[n]; var d1 = new bool[2 * n]; var d2 = new bool[2 * n];
    var board = new char[n][];
    for (int i = 0; i < n; i++) board[i] = Enumerable.Repeat('.', n).ToArray();
    void Place(int r)
    {
        if (r == n) { res.Add(board.Select(row => new string(row)).ToList()); return; }
        for (int c = 0; c < n; c++)
        {
            if (cols[c] || d1[r + c] || d2[r - c + n]) continue;      // attacked?
            cols[c] = d1[r + c] = d2[r - c + n] = true; board[r][c] = 'Q';
            Place(r + 1);
            cols[c] = d1[r + c] = d2[r - c + n] = false; board[r][c] = '.';
        }
    }
    Place(0);
    return res;
}
```

★ Interview Tip

Explain backtracking as a **decision tree**: each level is a decision, each branch a choice. Pruning cuts branches that cannot lead to a solution, which is what makes exponential problems feasible.

6. Searching and Binary Search

6.1 Linear Search

> ✎ **Definition**
>
> **Linear search** checks each element one by one until the target is found. It takes O(n) and works on unsorted data.

Check each element: O(n). Works on unsorted data.

```
int LinearSearch(int[] a, int target)
{
    for (int i = 0; i < a.Length; i++) if (a[i] == target) return i;
    return -1;
}
```

6.2 Binary Search

> ✎ **Definition**
>
> **Binary search** finds a target in **sorted** data by repeatedly halving the search range, comparing with the middle element each time. It takes O(log n).

On a **sorted** (or monotonic) search space, compare with the middle and discard half each step: **O(log n)**. 1 billion elements need only ~30 steps.

> ☼ **Simple Analogy**
>
> Looking up a word in a dictionary: open it in the middle, decide whether the word is in the left or right half, and repeat - you never read page by page.

```
int BinarySearch(int[] a, int target)
{
    int lo = 0, hi = a.Length - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;              // avoids (lo + hi) overflow
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

> ⚡ **Tricky Point**
>
> Three classic bugs: (1) `(lo + hi) / 2` overflows for large indices; (2) `while (lo < hi)` vs `lo <= hi` must match how you update bounds; (3) writing `lo = mid` with `while (lo < hi)` can loop forever when `hi = lo + 1`. Test with arrays of size 0, 1 and 2.

6.3 Lower Bound and Upper Bound

> **✎ Definition**
>
> **Lower bound** is the first index with value ≥ target, and **upper bound** is the first index with value > target. Both are binary-search variants for ranges and insertion points.

Find the **first index** where a condition becomes true - the most reusable binary-search template.

```
// First index with a[i] >= target (insertion point). Returns a.Length if none.
int LowerBound(int[] a, int target)
{
    int lo = 0, hi = a.Length;                    // half-open [lo, hi)
    while (lo < hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1; else hi = mid;
    }
    return lo;
```

```
}
// First index with a[i] > target
int UpperBound(int[] a, int target)
{
    int lo = 0, hi = a.Length;
    while (lo < hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] <= target) lo = mid + 1; else hi = mid;
    }
    return lo;
}
// Count of target = UpperBound - LowerBound;  first/last occurrence come for free
```

> **i Note**
>
> `Array.BinarySearch` returns the index if found; otherwise the **bitwise complement** of the insertion point (`~idx`). With duplicates it returns *any* matching index, not necessarily the first.

6.4 Search in Rotated Sorted Array

> **✎ Definition**
>
> Binary search on a sorted array that has been rotated. At each step one half is still sorted, which tells you which half can contain the target. O(log n).

```
int SearchRotated(int[] a, int target)
{
    int lo = 0, hi = a.Length - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid])                                    // left half sorted
        {
            if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        }
        else                                                    // right half sorted
        {
            if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}
// Minimum in rotated array: compare a[mid] with a[hi]
int FindMin(int[] a)
{
    int lo = 0, hi = a.Length - 1;
    while (lo < hi) { int mid = lo + (hi - lo) / 2; if (a[mid] > a[hi]) lo = mid + 1; else hi = mid; }
    return a[lo];
}
```

6.5 Binary Search on the Answer

> **✎ Definition**
>
> Binary search over the **range of possible answers** (not over an array), using a monotonic feasibility check, e.g. minimum capacity or maximum minimum distance.

When the answer is a number in a range and a check `Feasible(x)` is **monotonic** (false...false, true...true), binary search the answer itself.

```
// Koko eating bananas: minimum speed to finish all piles within h hours
int MinEatingSpeed(int[] piles, int h)
{
    int lo = 1, hi = piles.Max();
    while (lo < hi)
    {
        int speed = lo + (hi - lo) / 2;
        long hours = 0;
        foreach (int p in piles) hours += (p + speed - 1) / speed;   // ceil division
        if (hours <= h) hi = speed;                                  // feasible: try smaller
```

```
        else lo = speed + 1;
    }
    return lo;     // O(n log(max))
}
// Integer square root
int MySqrt(int x)
{
    long lo = 0, hi = x;
    while (lo < hi) { long mid = (lo + hi + 1) / 2; if (mid * mid <= x) lo = mid; else hi = mid - 1; }
    return (int)lo;
}
```

★ Interview Tip

Recognise binary search on the answer by phrases like "minimum maximum", "smallest capacity such that", "at least K", "minimise the largest". State: "the feasibility check is monotonic, so I binary search the answer range: O(n log range)."

6.6 Searching a 2-D Matrix

✎ Definition

Searching a sorted matrix, either by treating it as one flattened sorted array (O(log mn)) or by a staircase walk from a corner (O(m + n)).

```
// Rows and columns sorted: start top-right; O(m + n)
bool SearchMatrix(int[][] m, int target)
{
    int r = 0, c = m[0].Length - 1;
    while (r < m.Length && c >= 0)
    {
        if (m[r][c] == target) return true;
        if (m[r][c] > target) c--; else r++;
    }
    return false;
}
```

7. Sorting Algorithms

7.1 Overview

> ✎ **Definition**
>
> **Sorting** arranges elements in order. Algorithms differ in time and space complexity, **stability** (whether equal items keep their order) and whether they sort in place.

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Bubble sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Teaching only
Selection sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No	Minimum swaps
Insertion sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Great for small / nearly sorted data
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Linked lists, external sorting
Quick sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	Fastest in practice
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Guaranteed, in place
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes	Small integer range k
Radix sort	O(d(n + b))	O(d(n + b))	O(d(n + b))	O(n + b)	Yes	Fixed-length keys

Stable = equal elements keep their original relative order (important when sorting by multiple keys).

> **i Note**
>
> `Array.Sort` / `List<T>.Sort` in .NET use **introsort** (quick sort + heap sort fallback + insertion sort for small partitions): O(n log n) worst case, in place, **not stable**. LINQ `OrderBy` is **stable** (merge-based) but allocates.

7.2 Bubble, Selection and Insertion Sort

✎ Definition

Simple O(n^2) sorts. **Bubble** swaps adjacent out-of-order pairs, **selection** picks the minimum each pass, and **insertion** inserts each item into the sorted prefix (fast on nearly sorted data).

```
void BubbleSort(int[] a)
{
    for (int i = 0; i < a.Length - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < a.Length - 1 - i; j++)          // largest bubbles to the end
            if (a[j] > a[j + 1]) { (a[j], a[j + 1]) = (a[j + 1], a[j]); swapped = true; }
        if (!swapped) break;                                // already sorted: O(n) best case
    }
}
```

```
void SelectionSort(int[] a)
{
    for (int i = 0; i < a.Length - 1; i++)
    {
        int min = i;
        for (int j = i + 1; j < a.Length; j++) if (a[j] < a[min]) min = j;
        (a[i], a[min]) = (a[min], a[i]);                    // one swap per pass
    }
}
void InsertionSort(int[] a)
{
    for (int i = 1; i < a.Length; i++)
    {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }   // shift right
        a[j + 1] = key;                                          // like sorting playing cards
    }
}
```

☼ Simple Analogy

Insertion sort is how you sort cards in your hand: pick the next card and slide it left into its correct place among the cards already sorted.

7.3 Merge Sort (Divide and Conquer)

✎ Definition

Merge sort splits the array in half, sorts each half recursively and **merges** them. It is O(n log n) in all cases, stable, and needs O(n) extra space.

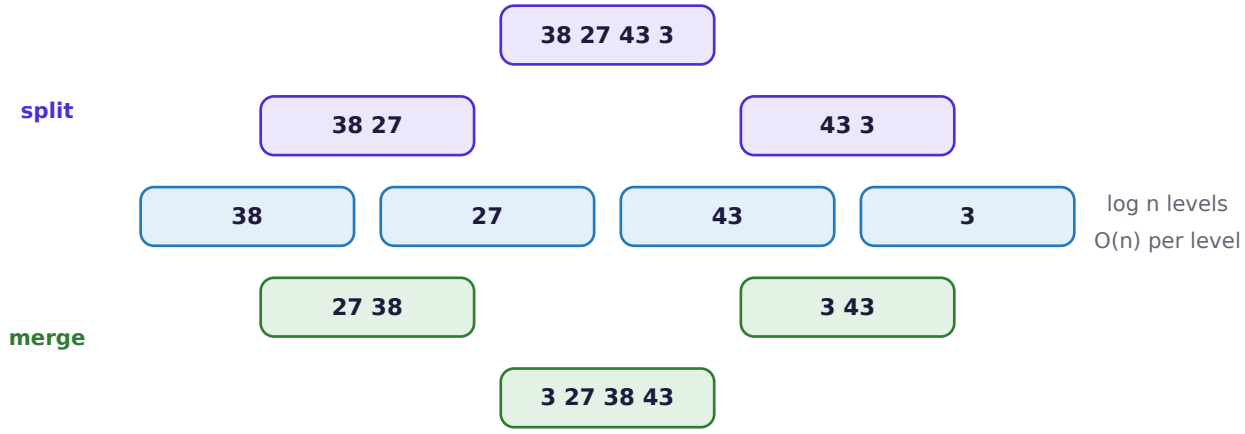


Figure 7.1 - Split until single elements, then merge sorted halves

```
void MergeSort(int[] a, int lo, int hi, int[] tmp)
{
    if (lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    MergeSort(a, lo, mid, tmp);
    MergeSort(a, mid + 1, hi, tmp);
    Merge(a, lo, mid, hi, tmp);
}
void Merge(int[] a, int lo, int mid, int hi, int[] tmp)
{
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];   // <= keeps it stable
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi) tmp[k++] = a[j++];
    Array.Copy(tmp, lo, a, lo, hi - lo + 1);
}
// usage: MergeSort(arr, 0, arr.Length - 1, new int[arr.Length]);
```

★ Interview Tip

Merge sort's merge step is reused in many problems: **counting inversions**, merging K sorted lists, and sorting linked lists in O(n log n) with O(1) extra space.

7.4 Quick Sort

✎ Definition

Quick sort picks a **pivot**, partitions elements into smaller and larger sides, and sorts each side recursively. O(n log n) on average, O(n^2) worst case, in place.

Pick a **pivot**, partition so smaller elements go left and larger go right, then recurse.

```
void QuickSort(int[] a, int lo, int hi)
{
    if (lo >= hi) return;
    int p = Partition(a, lo, hi);
    QuickSort(a, lo, p - 1);
    QuickSort(a, p + 1, hi);
}
int Partition(int[] a, int lo, int hi)                      // Lomuto scheme
{
    int r = Random.Shared.Next(lo, hi + 1);                 // random pivot avoids O(n^2) on sorted input
    (a[r], a[hi]) = (a[hi], a[r]);
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) { (a[i], a[j]) = (a[j], a[i]); i++; }
    (a[i], a[hi]) = (a[hi], a[i]);
    return i;
}
```

⚡ Tricky Point

Quick sort degrades to **O(n^2)** when the pivot is always the smallest/largest (e.g. first element on already-sorted data). Use a random pivot or median-of-three. Many equal elements also hurt Lomuto: use 3-way partitioning (Dutch National Flag).

Quickselect: k-th smallest in average O(n)

```
int KthSmallest(int[] a, int k)                             // k is 0-based
{
    int lo = 0, hi = a.Length - 1;
    while (true)
    {
        int p = Partition(a, lo, hi);
        if (p == k) return a[p];
        if (p < k) lo = p + 1; else hi = p - 1;
    }
}
```

7.5 Heap Sort and Counting Sort

✎ Definition

Heap sort builds a max-heap and repeatedly extracts the maximum, taking O(n log n) in place. **Counting sort** counts occurrences of small integer keys, taking O(n + k).

```
void HeapSort(int[] a)
{
    int n = a.Length;
    for (int i = n / 2 - 1; i >= 0; i--) SiftDown(a, i, n);      // build max-heap O(n)
    for (int end = n - 1; end > 0; end--)
    {
        (a[0], a[end]) = (a[end], a[0]);                          // move max to the end
        SiftDown(a, 0, end);
    }
}
```

```
void SiftDown(int[] a, int i, int n)
{
    while (true)
    {
        int largest = i, l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) return;
        (a[i], a[largest]) = (a[largest], a[i]);
        i = largest;
    }
}

int[] CountingSort(int[] a, int maxValue)                        // values in 0..maxValue
{
    var count = new int[maxValue + 1];
    foreach (int x in a) count[x]++;
    var res = new int[a.Length]; int k = 0;
    for (int v = 0; v <= maxValue; v++) while (count[v]-- > 0) res[k++] = v;
    return res;
}
```

7.6 Dutch National Flag (Sort 0s, 1s, 2s)

✎ Definition

The **Dutch National Flag** algorithm sorts an array of three values (0s, 1s, 2s) in one pass using three pointers (low, mid, high). O(n) time, O(1) space.

```
void SortColors(int[] a)
{
    int low = 0, mid = 0, high = a.Length - 1;
    while (mid <= high)
    {
        if (a[mid] == 0) { (a[low], a[mid]) = (a[mid], a[low]); low++; mid++; }
        else if (a[mid] == 1) mid++;
        else { (a[mid], a[high]) = (a[high], a[mid]); high--; }   // don't advance mid here!
    }
}
```

7.7 Custom Sorting in C#

✎ Definition

Sorting with custom order in C#: `Array.Sort`/`List.Sort` with a `Comparison` lambda or `IComparer`, and LINQ `OrderBy`/`ThenBy` (stable).

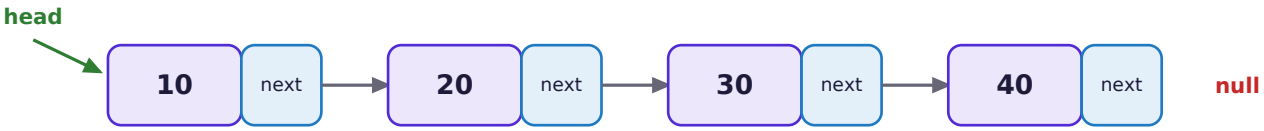
```
var people = new List<(string Name, int Age)> { ("Zara", 25), ("Ali", 30), ("Sam", 25) };
people.Sort((x, y) => x.Age != y.Age ? x.Age.CompareTo(y.Age) : string.CompareOrdinal(x.Name, y.Name));
int[][] intervals = { new[] { 5, 6 }, new[] { 1, 3 } };
Array.Sort(intervals, (x, y) => x[0].CompareTo(y[0]));        // sort intervals by start
var ordered = people.OrderBy(p => p.Age).ThenByDescending(p => p.Name).ToList();   // stable
string largestNumber = string.Concat(new[] { 3, 30, 34 }.Select(x => x.ToString())
                        .OrderBy(x => x, Comparer<string>.Create((a, b) => (b + a).CompareTo(a + b))));  // "34330"
```

8. Linked Lists

8.1 Structure

> **✎ Definition**
>
> A **linked list** is a chain of **nodes**, each holding a value and a reference to the next node (and to the previous one in a doubly linked list). Elements are not stored contiguously.

A linked list is a chain of **nodes**; each node stores a value and a reference to the next node (and to the previous one in a doubly linked list).



Nodes scattered in memory, linked by references. Insert/delete O(1) given a node; access O(n).

Figure 8.1 - A singly linked list

```
public class ListNode
{
    public int val;
    public ListNode? next;
    public ListNode(int val = 0, ListNode? next = null) { this.val = val; this.next = next; }
}
```

Array / List<T>	Linked list
Contiguous memory, cache-friendly	Nodes scattered in memory
O(1) access by index	O(n) access by index
O(n) insert/delete at front or middle	O(1) insert/delete when you hold the node
Fixed size (array) or amortised growth	Grows node by node, extra memory per node

> **i Note**
>
> .NET provides `LinkedList<T>` (doubly linked, with `AddFirst`, `AddLast`, `AddAfter`, `Remove(node)`), but interview problems use your own `ListNode` class.

8.2 Basic Operations

✎ Definition

The basic linked-list operations: traversal, insertion and deletion at the head, tail or a given node, and search, often simplified with a **dummy** (sentinel) head node.

```
ListNode? InsertAtHead(ListNode? head, int v) => new ListNode(v, head);

int Length(ListNode? head) { int n = 0; for (var c = head; c != null; c = c.next) n++; return n; }

ListNode? DeleteValue(ListNode? head, int v)
{
    var dummy = new ListNode(0, head);          // dummy head removes special cases
    var cur = dummy;
    while (cur.next != null)
    {
        if (cur.next.val == v) cur.next = cur.next.next;
        else cur = cur.next;
    }
    return dummy.next;
}
```

⚡ Tricky Point

Use a dummy (sentinel) node whenever the head might change (deletions, merges, insertions at the front). It removes most null-checks and "is this the head?" special cases.

8.3 Reverse a Linked List

✎ Definition

Reversing the direction of all **next** pointers, iteratively with three pointers (prev, curr, next) or recursively. O(n) time.

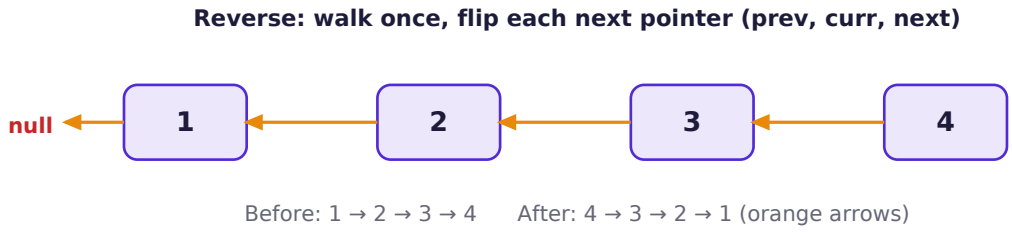


Figure 8.2 - Reversing pointers in one pass

```
ListNode? Reverse(ListNode? head)                // O(n) time, O(1) space
{
    ListNode? prev = null, cur = head;
    while (cur != null)
    {
        var next = cur.next;    // 1. save next
        cur.next = prev;        // 2. reverse the link
        prev = cur;             // 3. move prev forward
        cur = next;             // 4. move cur forward
    }
    return prev;                // new head
}
ListNode? ReverseRecursive(ListNode? head)
{
    if (head?.next == null) return head;
    var newHead = ReverseRecursive(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;             // O(n) stack space
}
```

8.4 Fast and Slow Pointers (Floyd)

> ✎ **Definition**
>
> **Floyd's tortoise and hare**: a slow pointer moves one step and a fast pointer two. Used to detect cycles, find a cycle's start, and find the middle of a list.

```
ListNode Middle(ListNode head)                   // slow moves 1, fast moves 2
{
    ListNode slow = head, fast = head;
    while (fast?.next != null) { slow = slow.next!; fast = fast.next.next!; }
    return slow;                                 // second middle for even length
}

bool HasCycle(ListNode? head)
{
    ListNode? slow = head, fast = head;
    while (fast?.next != null)
    {
        slow = slow!.next; fast = fast.next.next;
        if (slow == fast) return true;           // fast laps slow inside the cycle
    }
    return false;
}

ListNode? CycleStart(ListNode? head)
{
```

```
    ListNode? slow = head, fast = head;
    while (fast?.next != null)
    {
        slow = slow!.next; fast = fast.next.next;
        if (slow == fast)
        {
            slow = head;                          // distance math: restart one pointer
            while (slow != fast) { slow = slow!.next; fast = fast!.next; }
            return slow;
        }
    }
    return null;
}
```

> ☼ **Simple Analogy**
>
> Two runners on a circular track: the faster one will eventually lap the slower one. If the track has an end (no cycle), the fast runner simply reaches it.

8.5 Classic Problems

> ✎ **Definition**
>
> Standard linked-list interview problems: merge two sorted lists, remove the nth node from the end, palindrome check, intersection of two lists, and reordering.

```
ListNode? MergeTwoLists(ListNode? a, ListNode? b)
{
    var dummy = new ListNode(); var tail = dummy;
    while (a != null && b != null)
    {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = a ?? b;
    return dummy.next;
}

ListNode? RemoveNthFromEnd(ListNode head, int n)        // one pass with a gap of n
{
    var dummy = new ListNode(0, head);
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i <= n; i++) fast = fast.next!;
    while (fast != null) { fast = fast.next!; slow = slow.next!; }
    slow.next = slow.next!.next;
    return dummy.next;
}

bool IsPalindrome(ListNode head)                         // middle + reverse second half
{
    var second = Reverse(Middle(head));
    for (ListNode? p = head, q = second; q != null; p = p!.next, q = q.next)
        if (p!.val != q.val) return false;
    return true;
}
ListNode? GetIntersection(ListNode? a, ListNode? b)      // switch heads trick
{
    ListNode? p = a, q = b;
    while (p != q) { p = p == null ? b : p.next; q = q == null ? a : q.next; }
    return p;                                             // both travel lenA + lenB
}

ListNode? AddTwoNumbers(ListNode? l1, ListNode? l2)       // digits in reverse order
{
```

```
    var dummy = new ListNode(); var t = dummy; int carry = 0;
    while (l1 != null || l2 != null || carry != 0)
    {
        int s = (l1?.val ?? 0) + (l2?.val ?? 0) + carry;
        carry = s / 10; t.next = new ListNode(s % 10); t = t.next;
        l1 = l1?.next; l2 = l2?.next;
    }
    return dummy.next;
}
```

> **⚠ Watch Out**
>
> Linked list bugs are almost always about **losing references** (overwrite `next` before saving it) or **null dereferences** (`fast.next.next` when `fast.next` is null). Draw boxes and arrows on paper and update them step by step.

8.6 LRU Cache (Dictionary + Doubly Linked List)

> **✎ Definition**
>
> An **LRU** (Least Recently Used) cache evicts the item unused for the longest time. A dictionary plus a doubly linked list gives O(1) get and put.

```
public class LRUCache
{
    private readonly int _cap;
    private readonly Dictionary<int, LinkedListNode<(int key, int val)>> _map = new();
    private readonly LinkedList<(int key, int val)> _list = new();   // front = most recent
    public LRUCache(int capacity) => _cap = capacity;

    public int Get(int key)
    {
        if (!_map.TryGetValue(key, out var node)) return -1;
        _list.Remove(node); _list.AddFirst(node);                    // O(1) move to front
        return node.Value.val;
    }
    public void Put(int key, int value)
    {
        if (_map.TryGetValue(key, out var node)) _list.Remove(node);
        else if (_map.Count == _cap) { _map.Remove(_list.Last!.Value.key); _list.RemoveLast(); }
        _map[key] = _list.AddFirst((key, value));
    }
}
```

9. Stacks and Queues

9.1 Concepts

> ✎ **Definition**
>
> A **stack** is Last-In-First-Out (push and pop at the top). A **queue** is First-In-First-Out (enqueue at the back, dequeue from the front). Both operations are O(1).

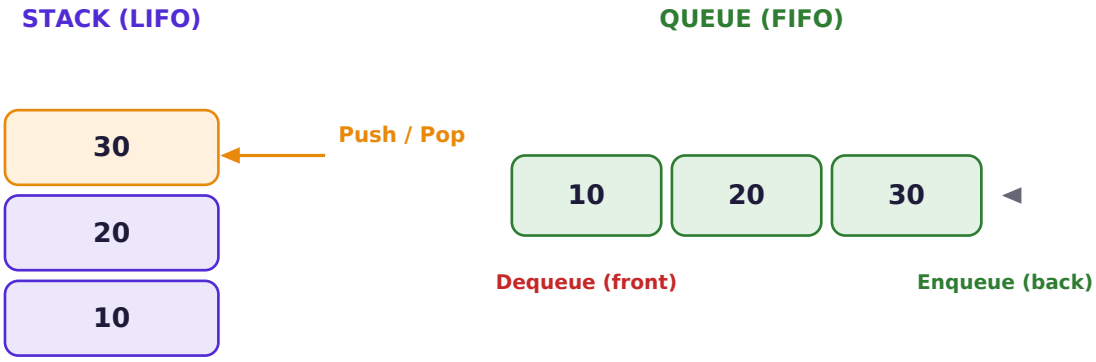


Figure 9.1 - Stack (LIFO) vs Queue (FIFO)

Stack (LIFO)	Queue (FIFO)
Last In, First Out	First In, First Out
`Push`, `Pop`, `Peek` - all O(1)	`Enqueue`, `Dequeue`, `Peek` - all O(1)
Undo, back button, recursion, DFS, parsing	Task scheduling, BFS, buffering, printers
C#: `Stack<T>`	C#: `Queue<T>`; deque: `LinkedList<T>`

> ☼ **Simple Analogy**
>
> Stack = a pile of plates (take from the top). Queue = a line at a ticket counter (first come, first served).

Implementing a stack and a queue with arrays

```
public class ArrayStack
{
    private int[] _data = new int[4]; private int _count;
    public void Push(int x)
    {
        if (_count == _data.Length) Array.Resize(ref _data, _data.Length * 2);   // amortised O(1)
        _data[_count++] = x;
    }
    public int Pop() => _count == 0 ? throw new InvalidOperationException("Empty") : _data[--_count];
    public int Peek() => _data[_count - 1];
    public bool IsEmpty => _count == 0;
}

public class CircularQueue                    // fixed capacity ring buffer
{
    private readonly int[] _d; private int _head, _count;
    public CircularQueue(int k) => _d = new int[k];
    public bool Enqueue(int x)
    {
        if (_count == _d.Length) return false;
        _d[(_head + _count++) % _d.Length] = x; return true;
    }
    public bool Dequeue() { if (_count == 0) return false; _head = (_head + 1) % _d.Length; _count--; return true; }
    public int Front() => _count == 0 ? -1 : _d[_head];
}
```

> ⚡ **Tricky Point**
>
> `Stack<T>.Pop()` / `Queue<T>.Dequeue()` on an empty collection throws `InvalidOperationException`. Check `Count > 0` or use `TryPop` / `TryDequeue` / `TryPeek`.

9.2 Stack Problems

> ✎ **Definition**
>
> Problems solved with a stack: matching brackets, evaluating expressions, undo history, simulating recursion, and min-stack designs.

Valid parentheses

```
bool IsValid(string s)
{
    var st = new Stack<char>();
    var pairs = new Dictionary<char, char> { [')'] = '(', [']'] = '[', ['}'] = '{' };
    foreach (char c in s)
    {
        if (pairs.TryGetValue(c, out char open))
        {
            if (st.Count == 0 || st.Pop() != open) return false;
        }
        else st.Push(c);
    }
    return st.Count == 0;        // leftover opens = invalid
}
```

Min stack (O(1) GetMin)

```
public class MinStack
{
    private readonly Stack<(int val, int min)> _st = new();
    public void Push(int v) => _st.Push((v, _st.Count == 0 ? v : Math.Min(v, _st.Peek().min)));
    public void Pop() => _st.Pop();
    public int Top() => _st.Peek().val;
    public int GetMin() => _st.Peek().min;
}
```

Evaluate Reverse Polish Notation

```
int EvalRPN(string[] tokens)
{
    var st = new Stack<int>();
    foreach (var t in tokens)
    {
        if (t is "+" or "-" or "*" or "/")
        {
            int b = st.Pop(), a = st.Pop();         // order matters: a op b
            st.Push(t switch { "+" => a + b, "-" => a - b, "*" => a * b, _ => a / b });
        }
        else st.Push(int.Parse(t));
    }
    return st.Pop();
}
```

9.3 Monotonic Stack

> **✎ Definition**
>
> A **monotonic stack** keeps its elements in increasing or decreasing order. It solves **next greater/smaller element** and histogram problems in O(n).

A stack kept in increasing or decreasing order. It answers "**next greater / smaller element**" questions in **O(n)** (each element is pushed and popped at most once).

```
int[] NextGreater(int[] a)                        // -1 if none
{
    var res = new int[a.Length]; Array.Fill(res, -1);
    var st = new Stack<int>();                     // indices, values decreasing
    for (int i = 0; i < a.Length; i++)
    {
        while (st.Count > 0 && a[st.Peek()] < a[i]) res[st.Pop()] = a[i];
        st.Push(i);
    }
    return res;
}
// a = {2, 1, 2, 4, 3} -> {4, 2, 4, -1, -1}

int[] DailyTemperatures(int[] t)                  // days until a warmer temperature
{
    var res = new int[t.Length]; var st = new Stack<int>();
    for (int i = 0; i < t.Length; i++)
    {
        while (st.Count > 0 && t[st.Peek()] < t[i]) { int j = st.Pop(); res[j] = i - j; }
        st.Push(i);
    }
    return res;
}

int LargestRectangleArea(int[] h)                 // histogram, O(n)
{
    var st = new Stack<int>(); int best = 0;
    for (int i = 0; i <= h.Length; i++)
    {
        int cur = i == h.Length ? 0 : h[i];       // sentinel 0 flushes the stack
        while (st.Count > 0 && h[st.Peek()] > cur)
        {
            int height = h[st.Pop()];
            int width = st.Count == 0 ? i : i - st.Peek() - 1;
            best = Math.Max(best, height * width);
        }
        st.Push(i);
    }
    return best;
}
```

★ Interview Tip

Recognise monotonic stack problems by "next greater", "previous smaller", "span", "temperatures", "histogram", "trapping rain water". Store **indices** (not values) so you can compute distances.

9.4 Queue Problems and Deque

> ✎ **Definition**
>
> Problems solved with a queue (BFS, scheduling, sliding windows) and a **deque**, a double-ended queue, for example for the sliding window maximum.

```
public class MyQueue                            // queue using two stacks, amortised O(1)
{
    private readonly Stack<int> _in = new(), _out = new();
    public void Push(int x) => _in.Push(x);
    public int Pop() { Move(); return _out.Pop(); }
    public int Peek() { Move(); return _out.Peek(); }
    private void Move() { if (_out.Count == 0) while (_in.Count > 0) _out.Push(_in.Pop()); }
}

int[] MaxSlidingWindow(int[] a, int k)          // monotonic deque, O(n)
{
    var dq = new LinkedList<int>();             // indices, values decreasing
    var res = new int[a.Length - k + 1];
    for (int i = 0; i < a.Length; i++)
    {
        if (dq.Count > 0 && dq.First!.Value <= i - k) dq.RemoveFirst();       // out of window
        while (dq.Count > 0 && a[dq.Last!.Value] <= a[i]) dq.RemoveLast();    // smaller are useless
        dq.AddLast(i);
        if (i >= k - 1) res[i - k + 1] = a[dq.First!.Value];
    }
    return res;
}
```

10. Trees and Binary Search Trees

10.1 Terminology

> **✎ Definition**
>
> Tree vocabulary: **root**, **parent/child**, **leaf**, **depth**, **height**, **subtree**, and types such as binary, full, complete, perfect and balanced trees.

A tree is a hierarchical structure of nodes connected by edges, with one **root** and no cycles. A **binary tree** has at most two children per node.

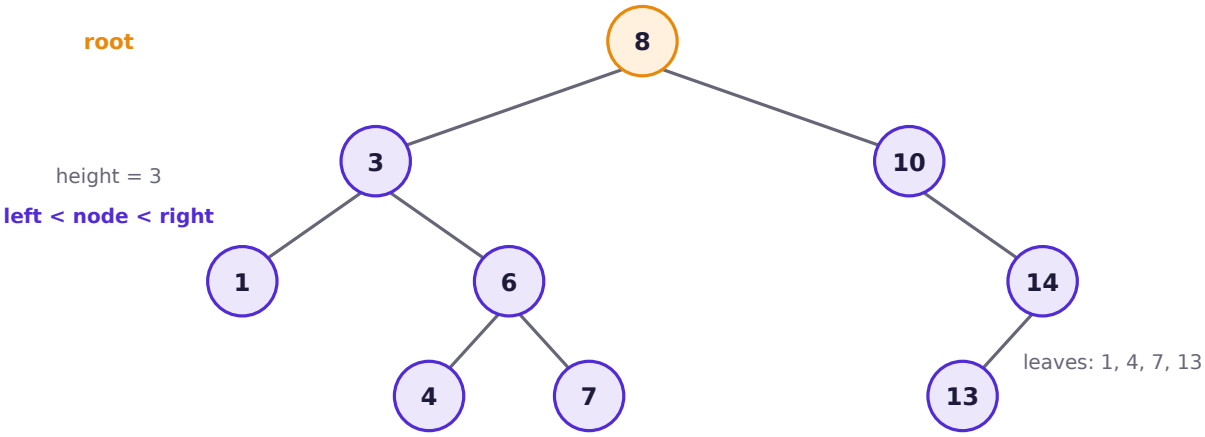


Figure 10.1 - A binary search tree

Term	Meaning
Root	Top node (no parent)
Leaf	Node with no children
Depth of a node	Edges from the root to the node
Height of a tree	Edges on the longest root-to-leaf path
Full binary tree	Every node has 0 or 2 children
Complete binary tree	All levels full except possibly the last, filled left to right (heaps)
Perfect binary tree	All internal nodes have 2 children and all leaves are at the same level
Balanced tree	Height is O(log n) (AVL, red-black)
BST	For every node: left subtree < node < right subtree

```
public class TreeNode
{
    public int val;
    public TreeNode? left, right;
    public TreeNode(int val = 0, TreeNode? left = null, TreeNode? right = null)
    { this.val = val; this.left = left; this.right = right; }
}
```

10.2 Traversals

> ✎ **Definition**
>
> Ways to visit every node of a tree: depth-first **pre-order**, **in-order** and **post-order**, and breadth-first **level-order**.

Pre-order (N L R)	8 3 1 6 4 7 10 14 13
In-order (L N R)	1 3 4 6 7 8 10 13 14 (sorted!)
Post-order (L R N)	1 4 7 6 3 13 14 10 8
Level-order (BFS)	8 | 3 10 | 1 6 14 | 4 7 13

Figure 10.2 - Traversal orders for the tree in Figure 10.1

```
void PreOrder(TreeNode? n, List<int> res)
{
    if (n == null) return;
    res.Add(n.val);            // Node
    PreOrder(n.left, res);     // Left
    PreOrder(n.right, res);    // Right
}
void InOrder(TreeNode? n, List<int> res)
{
    if (n == null) return;
    InOrder(n.left, res); res.Add(n.val); InOrder(n.right, res);     // L N R
}
void PostOrder(TreeNode? n, List<int> res)
{
    if (n == null) return;
    PostOrder(n.left, res); PostOrder(n.right, res); res.Add(n.val); // L R N
}

// Iterative in-order with an explicit stack (no recursion depth risk)
IList<int> InOrderIterative(TreeNode? root)
{
    var res = new List<int>(); var st = new Stack<TreeNode>(); var cur = root;
    while (cur != null || st.Count > 0)
    {
        while (cur != null) { st.Push(cur); cur = cur.left; }   // go left as far as possible
        cur = st.Pop();
        res.Add(cur.val);
        cur = cur.right;
    }
    return res;
}

// Level-order (BFS) grouped by level
IList<IList<int>> LevelOrder(TreeNode? root)
{
    var res = new List<IList<int>>();
    if (root == null) return res;
    var q = new Queue<TreeNode>(); q.Enqueue(root);
    while (q.Count > 0)
    {
        int size = q.Count;                       // nodes in the current level
        var level = new List<int>(size);
        for (int i = 0; i < size; i++)
        {
            var n = q.Dequeue(); level.Add(n.val);
            if (n.left != null) q.Enqueue(n.left);
            if (n.right != null) q.Enqueue(n.right);
        }
        res.Add(level);
    }
    return res;
}
```

⚡ Tricky Point

In-order traversal of a BST gives sorted order. This single fact solves many problems: validate a BST, k-th smallest element, convert BST to sorted list, find two nodes that were swapped.

10.3 Classic Recursive Tree Problems

> **✎ Definition**
>
> Tree problems solved naturally with recursion: maximum depth, same tree, symmetric tree, path sum, diameter, invert, and lowest common ancestor.

Most tree problems follow the pattern: **solve for left, solve for right, combine at the node.**

```
int MaxDepth(TreeNode? n) => n == null ? 0 : 1 + Math.Max(MaxDepth(n.left), MaxDepth(n.right));

TreeNode? Invert(TreeNode? n)
{
    if (n == null) return null;
    (n.left, n.right) = (Invert(n.right), Invert(n.left));
    return n;
}

bool IsSameTree(TreeNode? p, TreeNode? q) =>
    p == null || q == null ? p == q : p.val == q.val && IsSameTree(p.left, q.left) && IsSameTree(p.right, q.right);

bool IsSymmetric(TreeNode? root) => Mirror(root?.left, root?.right);
bool Mirror(TreeNode? a, TreeNode? b) =>
    a == null || b == null ? a == b : a.val == b.val && Mirror(a.left, b.right) && Mirror(a.right, b.left);

int Diameter(TreeNode? root)                      // longest path between any two nodes (edges)
{
    int best = 0;
    int Height(TreeNode? n)
    {
        if (n == null) return 0;
        int l = Height(n.left), r = Height(n.right);
        best = Math.Max(best, l + r);              // path through n
        return 1 + Math.Max(l, r);                 // height returned to parent
    }
    Height(root);
    return best;
}
bool IsBalanced(TreeNode? root)
{
    int Check(TreeNode? n)                         // -1 means unbalanced
    {
        if (n == null) return 0;
        int l = Check(n.left); if (l < 0) return -1;
        int r = Check(n.right); if (r < 0) return -1;
        return Math.Abs(l - r) > 1 ? -1 : 1 + Math.Max(l, r);
    }
    return Check(root) >= 0;
}

int MaxPathSum(TreeNode? root)                    // path may start/end anywhere
{
    int best = int.MinValue;
    int Gain(TreeNode? n)
    {
        if (n == null) return 0;
        int l = Math.Max(0, Gain(n.left)), r = Math.Max(0, Gain(n.right));   // drop negative branches
        best = Math.Max(best, n.val + l + r);
        return n.val + Math.Max(l, r);
    }
    Gain(root);
    return best;
}

bool HasPathSum(TreeNode? n, int target) =>
    n != null && (n.left == null && n.right == null ? n.val == target
                  : HasPathSum(n.left, target - n.val) || HasPathSum(n.right, target - n.val));
TreeNode? LowestCommonAncestor(TreeNode? root, TreeNode p, TreeNode q)   // binary tree
{
    if (root == null || root == p || root == q) return root;
    var l = LowestCommonAncestor(root.left, p, q);
    var r = LowestCommonAncestor(root.right, p, q);
    return l != null && r != null ? root : l ?? r;
}
```

> **★ Interview Tip**
>
> For "global answer" problems (diameter, max path sum), say: "the recursive function **returns** what the parent needs (a height/gain), while I **update a global best** with the value that uses both children."

10.4 Binary Search Tree Operations

> **✎ Definition**
>
> A **Binary Search Tree** keeps left subtree < node < right subtree, so search, insert and delete take O(h): O(log n) when balanced. In-order traversal gives sorted output.

Operation	Balanced BST	Skewed BST (worst)
Search / Insert / Delete	O(log n)	O(n)
Min / Max	O(log n)	O(n)
In-order traversal	O(n)	O(n)

```
TreeNode? Search(TreeNode? n, int key)
{
    while (n != null && n.val != key) n = key < n.val ? n.left : n.right;
    return n;
}

TreeNode Insert(TreeNode? n, int key)
{
    if (n == null) return new TreeNode(key);
    if (key < n.val) n.left = Insert(n.left, key);
    else if (key > n.val) n.right = Insert(n.right, key);
    return n;
}

TreeNode? Delete(TreeNode? n, int key)
{
    if (n == null) return null;
    if (key < n.val) n.left = Delete(n.left, key);
    else if (key > n.val) n.right = Delete(n.right, key);
    else
    {
        if (n.left == null) return n.right;        // 0 or 1 child
        if (n.right == null) return n.left;
        var succ = n.right;                        // 2 children: in-order successor
        while (succ.left != null) succ = succ.left;
        n.val = succ.val;
        n.right = Delete(n.right, succ.val);
    }
    return n;
}
bool IsValidBST(TreeNode? n, long lo = long.MinValue, long hi = long.MaxValue) =>
    n == null || (n.val > lo && n.val < hi && IsValidBST(n.left, lo, n.val) && IsValidBST(n.right, n.val, hi));

int KthSmallest(TreeNode root, int k)              // in-order, stop at k
{
    var st = new Stack<TreeNode>(); var cur = root;
    while (true)
    {
        while (cur != null) { st.Push(cur); cur = cur.left; }
        cur = st.Pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
}
```

> **⚡ Tricky Point**
>
> **Validate BST trap:** checking only `left.val < node.val < right.val` is wrong - every node in the left *subtree* must be smaller than the ancestor. Pass down `(low, high)` bounds, and use `long` bounds so nodes equal to `int.MinValue`/`int.MaxValue` work.

> **i Note**
>
> Inserting sorted data into a plain BST creates a linked-list-shaped tree (O(n) operations). Self-balancing trees (AVL, red-black) keep height O(log n). In C#, `SortedSet<T>` and `SortedDictionary<K,V>` are red-black trees.

10.5 Build Trees and Serialize

> **✎ Definition**
>
> Reconstructing a tree from traversal arrays (e.g. pre-order + in-order), and **serialization**: converting a tree to a string and back.

```
TreeNode? BuildTree(int[] preorder, int[] inorder)
{
    var idx = new Dictionary<int, int>();
    for (int i = 0; i < inorder.Length; i++) idx[inorder[i]] = i;
    int p = 0;
    TreeNode? Build(int lo, int hi)
    {
        if (lo > hi) return null;
        var root = new TreeNode(preorder[p++]);          // preorder: root first
        int mid = idx[root.val];
        root.left = Build(lo, mid - 1);
        root.right = Build(mid + 1, hi);
        return root;
    }
    return Build(0, inorder.Length - 1);
}

TreeNode? SortedArrayToBST(int[] a, int lo, int hi)      // balanced BST from sorted array
{
    if (lo > hi) return null;
    int mid = lo + (hi - lo) / 2;
    return new TreeNode(a[mid], SortedArrayToBST(a, lo, mid - 1), SortedArrayToBST(a, mid + 1, hi));
}
```

11. Heaps and Priority Queues

11.1 What is a Heap?

> ✎ **Definition**
>
> A **heap** is a complete binary tree where each parent is ≤ its children (min-heap) or ≥ them (max-heap). It is stored in an array and gives O(1) peek and O(log n) insert and remove.

A **binary heap** is a **complete binary tree** stored in an array where every parent is smaller than (min-heap) or larger than (max-heap) its children. The root is always the minimum (or maximum).

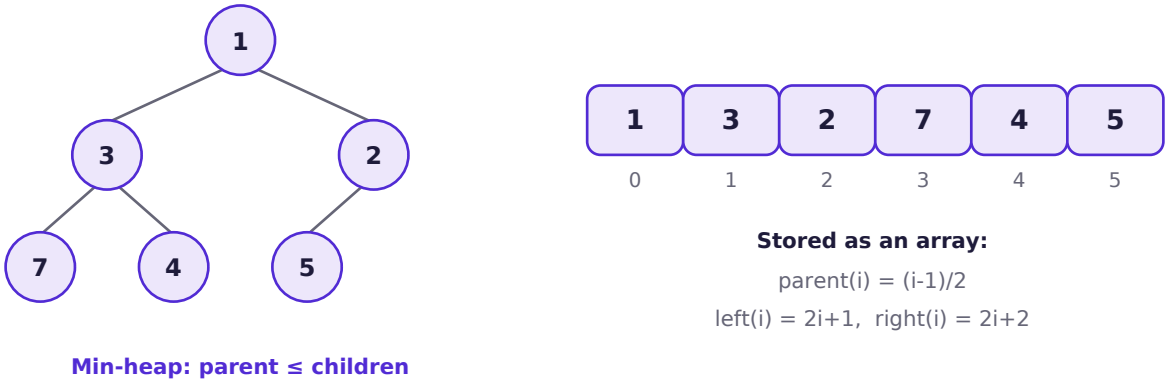


Figure 11.1 - A min-heap as a tree and as an array

Operation	Complexity
Peek min/max	O(1)
Insert (sift up)	O(log n)
Remove root (sift down)	O(log n)
Build heap from n items	O(n)
Search arbitrary item	O(n)

> ☼ **Simple Analogy**
>
> A hospital emergency room: patients don't leave in arrival order; the most critical one is always treated next. New arrivals are quickly slotted by severity.

11.2 PriorityQueue in C# (.NET 6+)

✎ Definition

`PriorityQueue<TElement, TPriority>` is .NET's built-in **min-heap**. Dequeue returns the lowest priority; use a custom comparer for max-heap behaviour.

```
var pq = new PriorityQueue<string, int>();          // element, priority (MIN-heap)
pq.Enqueue("low", 5); pq.Enqueue("urgent", 1); pq.Enqueue("mid", 3);
Console.WriteLine(pq.Peek());                       // urgent
while (pq.TryDequeue(out var item, out var pri)) Console.WriteLine($"{pri}:{item}");   // 1,3,5

var maxHeap = new PriorityQueue<int, int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));
maxHeap.Enqueue(7, 7);                               // element and priority often identical

var pq2 = new PriorityQueue<int[], (int dist, int id)>();   // tuple priorities compare lexicographically
var items = new[] { ("a", 3), ("b", 1) };
var pq3 = new PriorityQueue<string, int>(items);           // heapify in O(n)
int popped = maxHeap.EnqueueDequeue(4, 4);                  // push then pop efficiently
```

⚡ Tricky Point

C#'s `PriorityQueue` is a **min-heap** with no `Contains`, no `Remove(item)` and no priority update. For Dijkstra, push a new entry and skip stale ones when popped. Enumerating `UnorderedItems` is **not** sorted. It is also **not stable** for equal priorities.

11.3 Implementing a Min-Heap

> **✎ Definition**
>
> Building a min-heap by hand in an array, with **sift-up** after insert and **sift-down** after removing the root, using index arithmetic (children at 2i+1 and 2i+2).

```
public class MinHeap
{
    private readonly List<int> _a = new();
    public int Count => _a.Count;
    public int Peek() => _a[0];

    public void Push(int x)
    {
        _a.Add(x);
        int i = _a.Count - 1;
        while (i > 0)
        {
            int p = (i - 1) / 2;
            if (_a[p] <= _a[i]) break;
            (_a[p], _a[i]) = (_a[i], _a[p]);       // sift up
            i = p;
        }
    }
    public int Pop()
    {
        int top = _a[0];
        _a[0] = _a[^1]; _a.RemoveAt(_a.Count - 1);
        int i = 0;
        while (true)
        {
            int l = 2 * i + 1, r = l + 1, s = i;
            if (l < _a.Count && _a[l] < _a[s]) s = l;
            if (r < _a.Count && _a[r] < _a[s]) s = r;
            if (s == i) break;
            (_a[s], _a[i]) = (_a[i], _a[s]);       // sift down
            i = s;
        }
        return top;
    }
}
```

11.4 Top-K Pattern

> **✎ Definition**
>
> **Top-K** problems (the k largest, smallest or most frequent elements) are solved by keeping a heap of size k, giving O(n log k).

To find the **k largest** elements, keep a **min-heap of size k**: the root is the smallest of the top k, so anything smaller is discarded. O(n log k).

```
int FindKthLargest(int[] nums, int k)
{
    var pq = new PriorityQueue<int, int>();
    foreach (int x in nums)
    {
        pq.Enqueue(x, x);
        if (pq.Count > k) pq.Dequeue();            // remove smallest
    }
    return pq.Peek();
}
```

```
int[] TopKFrequent(int[] nums, int k)
{
    var freq = new Dictionary<int, int>();
    foreach (int x in nums) freq[x] = freq.GetValueOrDefault(x) + 1;
    var pq = new PriorityQueue<int, int>();        // element = number, priority = frequency
    foreach (var (num, f) in freq)
    {
        pq.Enqueue(num, f);
        if (pq.Count > k) pq.Dequeue();
    }
    var res = new int[k];
    for (int i = k - 1; i >= 0; i--) res[i] = pq.Dequeue();
    return res;
}

int[][] KClosest(int[][] points, int k)            // max-heap by distance, size k
{
    var pq = new PriorityQueue<int[], int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));
    foreach (var p in points)
    {
        pq.Enqueue(p, p[0] * p[0] + p[1] * p[1]);
        if (pq.Count > k) pq.Dequeue();
    }
    return Enumerable.Range(0, k).Select(_ => pq.Dequeue()).ToArray();
}
```

★ Interview Tip

"Top K" -> heap of size K (O(n log k)). "K-th element once" -> quickselect (average O(n)). "Stream of data" -> heap. "Range of small integers" -> bucket sort (O(n)).

11.5 Merge K Sorted Lists

✎ Definition

Merging k sorted lists into one sorted list using a min-heap of the current head of each list. O(N log k).

```
ListNode? MergeKLists(ListNode?[] lists)
{
    var pq = new PriorityQueue<ListNode, int>();
    foreach (var l in lists) if (l != null) pq.Enqueue(l, l.val);
    var dummy = new ListNode(); var tail = dummy;
    while (pq.TryDequeue(out var node, out _))
    {
        tail.next = node; tail = node;
        if (node.next != null) pq.Enqueue(node.next, node.next.val);
    }
    return dummy.next;              // O(N log k), N = total nodes
}
```

11.6 Median of a Data Stream (Two Heaps)

✎ Definition

Keeping a running median with **two heaps**: a max-heap for the lower half and a min-heap for the upper half, balanced so the median is always at the top.

```
public class MedianFinder
{
    private readonly PriorityQueue<int, int> _low =                    // max-heap: smaller half
        new(Comparer<int>.Create((a, b) => b.CompareTo(a)));
    private readonly PriorityQueue<int, int> _high = new();            // min-heap: larger half

    public void AddNum(int x)
    {
        _low.Enqueue(x, x);
        int m = _low.Dequeue(); _high.Enqueue(m, m);                    // move largest of low to high
        if (_high.Count > _low.Count) { int t = _high.Dequeue(); _low.Enqueue(t, t); }   // rebalance
    }
    public double FindMedian() =>
        _low.Count > _high.Count ? _low.Peek() : (_low.Peek() + (double)_high.Peek()) / 2;
}
```

⚡ Tricky Point

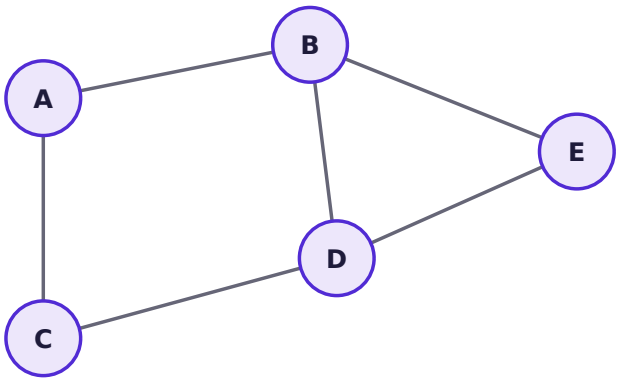
`(a + b) / 2` with two large ints overflows and also does **integer division**. Cast to `double` (or `long`) first.

12. Graphs

12.1 Terminology and Representation

> **✎ Definition**
>
> A **graph** is a set of **vertices** connected by **edges**, which may be directed or undirected, weighted or unweighted. It is stored as an **adjacency list** or an **adjacency matrix**.

A graph G = (V, E) is a set of **vertices** connected by **edges**. Edges may be directed or undirected, weighted or unweighted. Trees are special graphs (connected, no cycles, V-1 edges).



Adjacency list

A: B, C
B: A, D, E
C: A, D
D: B, C, E
E: B, D

Space O(V + E)
Matrix: O(V²), O(1) edge check

Figure 12.1 - An undirected graph and its adjacency list

Representation	Space	Check edge (u,v)	Iterate neighbours	Best for
Adjacency list	O(V + E)	O(deg)	O(deg)	Sparse graphs (most problems)
Adjacency matrix	O(V^2)	O(1)	O(V)	Dense graphs, Floyd-Warshall
Edge list	O(E)	O(E)	O(E)	Kruskal's MST, Bellman-Ford

```
// Build an adjacency list from an edge list (undirected)
List<int>[] BuildGraph(int n, int[][] edges)
{
    var g = new List<int>[n];
    for (int i = 0; i < n; i++) g[i] = new List<int>();
    foreach (var e in edges) { g[e[0]].Add(e[1]); g[e[1]].Add(e[0]); }   // directed: add one side only
    return g;
}
var weighted = new List<(int to, int w)>[n];                              // weighted graph
```

> **☼ Simple Analogy**
>
> A graph is a city map: vertices are junctions, edges are roads (one-way = directed), and weights are distances or travel times.

12.2 Breadth-First Search (BFS)

> **✎ Definition**
>
> **BFS** explores a graph **level by level** using a queue. It finds the shortest path in unweighted graphs. O(V + E).

Explores level by level using a **queue**. In an **unweighted** graph, BFS finds the **shortest path** (fewest edges).

BFS (Queue): A → B C → D E
level by level, shortest path (unweighted)

DFS (Stack/recursion): A → B → D → C ... E
go deep first, backtrack

Both visit every vertex and edge once: O(V + E). Always mark visited to avoid infinite loops.

Figure 12.2 - BFS vs DFS

```
int[] BfsDistances(List<int>[] g, int src)
{
    var dist = new int[g.Length]; Array.Fill(dist, -1);
    var q = new Queue<int>();
    dist[src] = 0; q.Enqueue(src);
    while (q.Count > 0)
    {
        int u = q.Dequeue();
        foreach (int v in g[u])
            if (dist[v] == -1)                    // mark visited WHEN ENQUEUING
            {
                dist[v] = dist[u] + 1;
                q.Enqueue(v);
            }
    }
    return dist;                                   // O(V + E)
}
```

> **⚡ Tricky Point**
>
> Mark a node as visited **when you enqueue it**, not when you dequeue it. Otherwise the same node can be enqueued many times, which blows up time and memory on dense graphs and grids.

12.3 Depth-First Search (DFS)

> **✎ Definition**
>
> **DFS** explores as **deep** as possible before backtracking, using recursion or a stack. Used for connectivity, cycle detection, components and topological order. O(V + E).

Goes as deep as possible before backtracking, using **recursion** or an explicit **stack**.

```
void Dfs(List<int>[] g, int u, bool[] visited)
{
    visited[u] = true;
    foreach (int v in g[u]) if (!visited[v]) Dfs(g, v, visited);
}

int CountComponents(int n, int[][] edges)
{
    var g = BuildGraph(n, edges); var visited = new bool[n]; int count = 0;
    for (int i = 0; i < n; i++) if (!visited[i]) { count++; Dfs(g, i, visited); }
    return count;
}
```

12.4 Grid Problems (Implicit Graphs)

✎ Definition

Treating a 2-D grid as an **implicit graph**, where each cell is a node connected to its neighbours. Used for flood fill, number of islands and shortest path in a maze.

Each cell is a vertex; neighbours are the 4 (or 8) adjacent cells.

```
int NumIslands(char[][] grid)
{
    int rows = grid.Length, cols = grid[0].Length, count = 0;
    int[] dr = { 1, -1, 0, 0 }, dc = { 0, 0, 1, -1 };
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
        {
            if (grid[r][c] != '1') continue;
            count++;
            var q = new Queue<(int, int)>(); q.Enqueue((r, c)); grid[r][c] = '0';   // sink it
            while (q.Count > 0)
            {
                var (cr, cc) = q.Dequeue();
                for (int d = 0; d < 4; d++)
                {
                    int nr = cr + dr[d], nc = cc + dc[d];
                    if (nr < 0 || nc < 0 || nr >= rows || nc >= cols || grid[nr][nc] != '1') continue;
                    grid[nr][nc] = '0'; q.Enqueue((nr, nc));
                }
            }
        }
    return count;    // O(rows * cols)
}
```

Multi-source BFS (rotting oranges)

```
int OrangesRotting(int[][] g)
{
    var q = new Queue<(int r, int c)>(); int fresh = 0, minutes = 0;
    for (int r = 0; r < g.Length; r++)
        for (int c = 0; c < g[0].Length; c++)
            if (g[r][c] == 2) q.Enqueue((r, c)); else if (g[r][c] == 1) fresh++;
    int[][] dirs = { new[] { 1, 0 }, new[] { -1, 0 }, new[] { 0, 1 }, new[] { 0, -1 } };
    while (q.Count > 0 && fresh > 0)
    {
        for (int i = q.Count; i > 0; i--)                       // one level = one minute
        {
            var (r, c) = q.Dequeue();
            foreach (var d in dirs)
            {
                int nr = r + d[0], nc = c + d[1];
                if (nr < 0 || nc < 0 || nr >= g.Length || nc >= g[0].Length || g[nr][nc] != 1) continue;
                g[nr][nc] = 2; fresh--; q.Enqueue((nr, nc));
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

⚠ Watch Out

Recursive DFS on a 1000 x 1000 grid can reach depth 10^6 and crash with `StackOverflowException`. Use BFS or an iterative DFS with `Stack<T>` for large grids.

12.5 Cycle Detection

> **✎ Definition**
>
> Checking whether a graph has a **cycle**: DFS with recursion-stack colours for directed graphs, and parent tracking or Union-Find for undirected graphs.

```
// Undirected: a visited neighbour that is not the parent means a cycle
bool HasCycleUndirected(List<int>[] g, int u, int parent, bool[] seen)
{
    seen[u] = true;
    foreach (int v in g[u])
    {
        if (!seen[v]) { if (HasCycleUndirected(g, v, u, seen)) return true; }
        else if (v != parent) return true;
    }
    return false;
}

// Directed: three colours - 0 unvisited, 1 in current path, 2 done
bool HasCycleDirected(List<int>[] g)
{
    var color = new int[g.Length];
    bool Visit(int u)
    {
        color[u] = 1;
        foreach (int v in g[u])
        {
            if (color[v] == 1) return true;            // back edge -> cycle
            if (color[v] == 0 && Visit(v)) return true;
        }
        color[u] = 2;
        return false;
    }
    for (int i = 0; i < g.Length; i++) if (color[i] == 0 && Visit(i)) return true;
    return false;
}
```

12.6 Topological Sort (Kahn's Algorithm)

> **✎ Definition**
>
> **Topological sort** orders the vertices of a directed acyclic graph so every edge goes from earlier to later. **Kahn's algorithm** repeatedly removes nodes with in-degree 0.

An ordering of a **DAG** (directed acyclic graph) where every edge u -> v puts u before v. Used for course schedules, build systems, task dependencies.

```
int[] TopoSort(int n, int[][] prerequisites)           // [course, prereq]
{
    var g = new List<int>[n]; var indeg = new int[n];
    for (int i = 0; i < n; i++) g[i] = new List<int>();
    foreach (var p in prerequisites) { g[p[1]].Add(p[0]); indeg[p[0]]++; }
    var q = new Queue<int>();
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.Enqueue(i);
    var order = new List<int>();
    while (q.Count > 0)
    {
        int u = q.Dequeue(); order.Add(u);
        foreach (int v in g[u]) if (--indeg[v] == 0) q.Enqueue(v);
    }
    return order.Count == n ? order.ToArray() : Array.Empty<int>();   // empty = cycle exists
}
```

12.7 Shortest Paths

> **✎ Definition**
>
> Algorithms for the minimum-cost path: **BFS** (unweighted), **Dijkstra** (non-negative weights, using a priority queue) and **Bellman-Ford** (handles negative weights).

Algorithm	Graph type	Complexity
BFS	Unweighted	O(V + E)
0-1 BFS (deque)	Weights 0 or 1	O(V + E)
Dijkstra	Non-negative weights	O((V + E) log V)
Bellman-Ford	Negative weights allowed, detects negative cycles	O(V * E)
Floyd-Warshall	All pairs	O(V^3)

```
int[] Dijkstra(List<(int to, int w)>[] g, int src)
{
    var dist = new int[g.Length]; Array.Fill(dist, int.MaxValue);
    var pq = new PriorityQueue<int, int>();
    dist[src] = 0; pq.Enqueue(src, 0);
    while (pq.TryDequeue(out int u, out int d))
    {
        if (d > dist[u]) continue;                     // stale entry: skip
        foreach (var (v, w) in g[u])
        {
            if (dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                pq.Enqueue(v, dist[v]);
            }
        }
    }
    return dist;
}
int[,] FloydWarshall(int[,] d, int n)                  // d[i,j] = weight or INF/2
{
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i, k] + d[k, j] < d[i, j]) d[i, j] = d[i, k] + d[k, j];
    return d;
}
```

> **⚡ Tricky Point**
>
> Dijkstra **fails with negative edge weights** (a settled node might later get a shorter path). Use Bellman-Ford. In Floyd-Warshall, use `INF = int.MaxValue / 2` so `INF + INF` doesn't overflow.

12.8 Minimum Spanning Tree (Kruskal)

> **✎ Definition**
>
> A **Minimum Spanning Tree** connects all vertices with the smallest total edge weight. **Kruskal's algorithm** adds the cheapest edges in order, using Union-Find to avoid cycles.

Connect all vertices with minimum total edge weight: sort edges, add each edge if it doesn't form a cycle (checked with Union-Find, Chapter 13).

```
int Kruskal(int n, List<(int u, int v, int w)> edges)
{
    edges.Sort((a, b) => a.w.CompareTo(b.w));
    var uf = new UnionFind(n); int total = 0, used = 0;
    foreach (var (u, v, w) in edges)
        if (uf.Union(u, v)) { total += w; if (++used == n - 1) break; }
    return used == n - 1 ? total : -1;                    // -1: graph not connected
}
```

Prim's algorithm is the alternative: grow the tree from a vertex using a priority queue of edges.

13. Tries, Union-Find and Range Query Structures

13.1 Trie (Prefix Tree)

> ✎ **Definition**
>
> A **Trie** (prefix tree) stores strings character by character along tree paths, so word insert, search and **prefix** queries take O(length of the word).

A trie stores strings character by character; words with the same prefix share a path. Insert and search take **O(L)** where L is the word length - independent of how many words are stored.

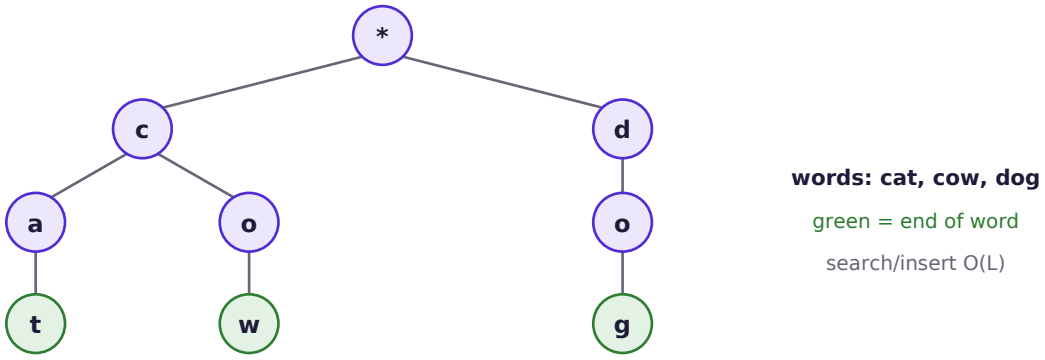


Figure 13.1 - A trie containing "cat", "cow" and "dog"

```
public class Trie
{
    private class Node
    {
        public readonly Node?[] Next = new Node?[26];
        public bool IsEnd;
    }
    private readonly Node _root = new();

    public void Insert(string word)
    {
        var cur = _root;
        foreach (char c in word) cur = cur.Next[c - 'a'] ??= new Node();
        cur.IsEnd = true;
    }
    public bool Search(string word) => Find(word)?.IsEnd == true;
    public bool StartsWith(string prefix) => Find(prefix) != null;

    private Node? Find(string s)
    {
        var cur = _root;
        foreach (char c in s) { cur = cur.Next[c - 'a']; if (cur == null) return null; }
        return cur;
    }
}
```

Uses: autocomplete, spell checkers, IP routing (longest prefix match), word search in a grid, XOR maximisation (bitwise trie). For large alphabets use `Dictionary<char, Node>` instead of an array of 26.

> **⚡ Tricky Point**
>
> `Search("ca")` must return false even though "ca" is a path in the trie - that's what the `IsEnd` flag is for. `StartsWith("ca")` returns true.

13.2 Union-Find (Disjoint Set Union)

> **✎ Definition**
>
> **Union-Find** (Disjoint Set Union) tracks which elements are in the same group, with near-O(1) **find** and **union** using path compression and union by rank.

Tracks elements partitioned into disjoint sets with two operations: **Find** (which set?) and **Union** (merge two sets). With **path compression** and **union by rank/size**, both are almost O(1) (inverse Ackermann).

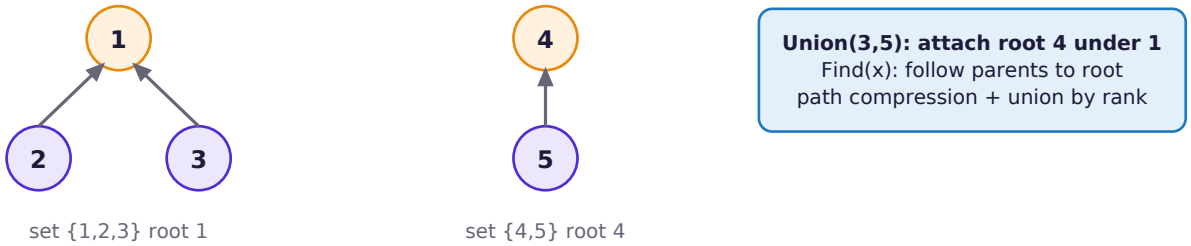


Figure 13.2 - Two sets represented as trees

```
public class UnionFind
{
    private readonly int[] _parent, _size;
    public int Components { get; private set; }
    public UnionFind(int n)
    {
        _parent = new int[n]; _size = new int[n]; Components = n;
        for (int i = 0; i < n; i++) { _parent[i] = i; _size[i] = 1; }
    }
    public int Find(int x)
    {
        while (_parent[x] != x)
        {
            _parent[x] = _parent[_parent[x]];      // path halving (compression)
            x = _parent[x];
        }
        return x;
    }
    public bool Union(int a, int b)
    {
        int ra = Find(a), rb = Find(b);
        if (ra == rb) return false;                // already connected (cycle!)
        if (_size[ra] < _size[rb]) (ra, rb) = (rb, ra);
        _parent[rb] = ra; _size[ra] += _size[rb]; // attach smaller under larger
        Components--;
        return true;
    }
    public bool Connected(int a, int b) => Find(a) == Find(b);
}
```

Uses: number of connected components, detecting cycles in undirected graphs, Kruskal's MST, accounts merge, redundant connection, grouping equivalent items (e.g. linking transactions that share a card or RRN).

> **★ Interview Tip**
>
> "When Union-Find over DFS?" - When edges arrive **dynamically** (online connectivity queries) or you need to repeatedly merge groups; DFS/BFS is fine for a static graph traversed once.

13.3 Fenwick Tree (Binary Indexed Tree)

> **✎ Definition**
>
> A **Fenwick Tree** (Binary Indexed Tree) is a compact array structure for **prefix sums with point updates**, both in O(log n).

Supports **point update** and **prefix sum** queries in O(log n) with a tiny implementation.

```
public class Fenwick
{
    private readonly long[] _t;
    public Fenwick(int n) => _t = new long[n + 1];             // 1-indexed internally
    public void Add(int i, long delta) { for (i++; i < _t.Length; i += i & -i) _t[i] += delta; }
    public long PrefixSum(int i) { long s = 0; for (i++; i > 0; i -= i & -i) s += _t[i]; return s; }  // sum [0..i]
    public long RangeSum(int l, int r) => PrefixSum(r) - (l > 0 ? PrefixSum(l - 1) : 0);
}
```

> **i Note**
>
> `i & -i` isolates the lowest set bit - the key trick behind Fenwick trees.

13.4 Segment Tree

> **✎ Definition**
>
> A **Segment Tree** is a binary tree over array ranges that answers range queries (sum, min, max) and handles updates in O(log n).

Supports range queries (sum, min, max, gcd) and updates in O(log n). More flexible than Fenwick (any associative operation, lazy range updates).

```
public class SegmentTree                                    // range sum, point update
{
    private readonly long[] _t; private readonly int _n;
    public SegmentTree(int[] a) { _n = a.Length; _t = new long[4 * _n]; Build(a, 1, 0, _n - 1); }
    private void Build(int[] a, int node, int l, int r)
    {
        if (l == r) { _t[node] = a[l]; return; }
        int m = (l + r) / 2;
        Build(a, 2 * node, l, m); Build(a, 2 * node + 1, m + 1, r);
        _t[node] = _t[2 * node] + _t[2 * node + 1];
    }
    public void Update(int idx, int val, int node = 1, int l = 0, int r = -1)
    {
        if (r == -1) r = _n - 1;
        if (l == r) { _t[node] = val; return; }
        int m = (l + r) / 2;
        if (idx <= m) Update(idx, val, 2 * node, l, m); else Update(idx, val, 2 * node + 1, m + 1, r);
        _t[node] = _t[2 * node] + _t[2 * node + 1];
    }
    public long Query(int ql, int qr, int node = 1, int l = 0, int r = -1)
    {
        if (r == -1) r = _n - 1;
        if (qr < l || r < ql) return 0;                         // no overlap
        if (ql <= l && r <= qr) return _t[node];                // full overlap
        int m = (l + r) / 2;
        return Query(ql, qr, 2 * node, l, m) + Query(ql, qr, 2 * node + 1, m + 1, r);
    }
}
```

Need	Use
Static array, many range sums	Prefix sum array (O(1) query)
Point updates + prefix/range sums	Fenwick tree
Range min/max, range updates, custom merges	Segment tree (with lazy propagation)
Sorted keys with rank/floor/ceiling	`SortedSet<T>` (`GetViewBetween`, `Min`, `Max`)

14. Greedy Algorithms and Intervals

14.1 The Greedy Idea

> ✎ **Definition**
>
> A **greedy algorithm** makes the **locally best choice** at each step and never revisits it. It gives the right answer only when the problem has the greedy-choice property.

A greedy algorithm makes the **locally best choice** at each step, hoping it leads to a global optimum. It works only when the problem has the **greedy-choice property** (a local optimum is part of some global optimum) and **optimal substructure**.

> ☼ **Simple Analogy**
>
> Giving change with Indian coins (10, 5, 2, 1): always take the largest coin that fits. This works for standard coin systems, but for coins {1, 3, 4} and amount 6, greedy gives 4+1+1 (3 coins) while the optimum is 3+3 (2 coins). That's when you need Dynamic Programming.

> ★ **Interview Tip**
>
> Before using greedy, try to find a **counter-example**. If you can justify it with an exchange argument ("swapping any choice for the greedy choice never makes it worse"), greedy is safe.

14.2 Classic Greedy Problems

✎ Definition

Well-known problems with correct greedy solutions: activity selection, jump game, gas station, minimum coins (for canonical coin systems), and task scheduling.

```
// Activity selection / max non-overlapping intervals: sort by END time
int MaxNonOverlapping(int[][] intervals)
{
    Array.Sort(intervals, (a, b) => a[1].CompareTo(b[1]));
    int count = 0, end = int.MinValue;
    foreach (var iv in intervals)
        if (iv[0] >= end) { count++; end = iv[1]; }
    return count;           // erase-overlap answer = n - count
}
// Jump game: can you reach the last index?
bool CanJump(int[] nums)
{
    int reach = 0;
    for (int i = 0; i < nums.Length; i++)
    {
        if (i > reach) return false;
        reach = Math.Max(reach, i + nums[i]);
    }
    return true;
}

// Minimum jumps (BFS-like levels)
int Jump(int[] nums)
{
    int jumps = 0, curEnd = 0, farthest = 0;
    for (int i = 0; i < nums.Length - 1; i++)
    {
        farthest = Math.Max(farthest, i + nums[i]);
        if (i == curEnd) { jumps++; curEnd = farthest; }
    }
    return jumps;
}

// Gas station
int CanCompleteCircuit(int[] gas, int[] cost)
{
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.Length; i++)
    {
```

```
        total += gas[i] - cost[i]; tank += gas[i] - cost[i];
        if (tank < 0) { start = i + 1; tank = 0; }       // can't start anywhere before i+1
    }
    return total >= 0 ? start : -1;
}

// Best time to buy and sell stock (one transaction)
int MaxProfit(int[] prices)
{
    int minPrice = int.MaxValue, best = 0;
    foreach (int p in prices) { minPrice = Math.Min(minPrice, p); best = Math.Max(best, p - minPrice); }
    return best;
}
```

14.3 Interval Problems

> **✎ Definition**
>
> Problems on ranges [start, end] (merge overlapping intervals, insert an interval, count meeting rooms, find the minimum to remove), usually solved by sorting on start or end.

Almost every interval problem starts with **sorting by start (or end) time**.

```
int[][] Merge(int[][] intervals)
{
    Array.Sort(intervals, (a, b) => a[0].CompareTo(b[0]));
    var res = new List<int[]>();
    foreach (var iv in intervals)
    {
        if (res.Count > 0 && iv[0] <= res[^1][1])
            res[^1][1] = Math.Max(res[^1][1], iv[1]);        // overlap: extend
        else res.Add(iv);
    }
    return res.ToArray();
}
// [[1,3],[2,6],[8,10],[15,18]] -> [[1,6],[8,10],[15,18]]

int MinMeetingRooms(int[][] meetings)                       // max simultaneous meetings
{
    var starts = meetings.Select(m => m[0]).OrderBy(x => x).ToArray();
    var ends = meetings.Select(m => m[1]).OrderBy(x => x).ToArray();
    int rooms = 0, best = 0, e = 0;
    foreach (int s in starts)
    {
        while (e < ends.Length && ends[e] <= s) { e++; rooms--; }
        rooms++; best = Math.Max(best, rooms);
    }
    return best;
}
```

> **⚡ Tricky Point**
>
> Clarify whether touching intervals overlap: is `[1,2]` and `[2,3]` a conflict? Merge-intervals problems usually treat them as overlapping (`<=`); meeting-room problems usually allow a meeting to start when another ends (`<`). One character changes the answer.

15. Dynamic Programming

15.1 What is DP?

> ✎ **Definition**
>
> **Dynamic Programming** solves problems with **overlapping sub-problems** and **optimal substructure** by solving each sub-problem once and storing the result.

Dynamic Programming solves problems by breaking them into **overlapping sub-problems**, solving each **once**, and storing the result. It applies when a problem has:

- **Optimal substructure:** the optimal answer is built from optimal answers of sub-problems.
- **Overlapping sub-problems:** the same sub-problems are needed again and again (see Figure 5.1).

> ☼ **Simple Analogy**
>
> If someone asks you "what is 1+1+1+1+1?" you count 5. If they add "+1", you don't recount - you remember 5 and answer 6. DP is remembering answers you've already computed.

Top-down (memoization)	Bottom-up (tabulation)
Recursion + cache	Iterative loops filling a table
Computes only needed states	Computes all states in order
Easier to write from the recurrence	No recursion depth risk, often faster
Uses call stack space	Easy to optimise space (rolling arrays)

15.2 The DP Recipe

> ✎ **Definition**
>
> A step-by-step method for DP: define the **state**, write the **recurrence**, set the **base cases**, choose top-down (memo) or bottom-up (table), then optimise space.

1. **Define the state:** what does `dp[i]` (or `dp[i][j]`) mean, in words? 2. **Write the transition (recurrence):** how is `dp[i]` computed from smaller states? 3. **Base cases:** the smallest states with known answers. 4. **Order of computation:** make sure states are computed before they're used. 5. **Answer location:** `dp[n]`, `dp[n-1][m-1]`, or the max over all states? 6. **Optimise space** if each row only depends on the previous one.

> ★ **Interview Tip**
>
> In interviews, say the state definition out loud first ("dp[i] is the maximum money robbing houses 0..i"). If the state is clear, the recurrence usually follows.

15.3 1-D DP

✎ Definition

DP where the state is a single index, such as climbing stairs, house robber, decode ways and longest increasing subsequence. Often reducible to O(1) space.

```
// Climbing stairs: 1 or 2 steps. dp[i] = dp[i-1] + dp[i-2]
int ClimbStairs(int n)
{
    if (n <= 2) return n;
    int a = 1, b = 2;                          // space-optimised: only two previous values
    for (int i = 3; i <= n; i++) (a, b) = (b, a + b);
    return b;
}

// House robber: dp[i] = max(dp[i-1], dp[i-2] + nums[i])
```

```
int Rob(int[] nums)
{
    int prev2 = 0, prev1 = 0;
    foreach (int x in nums) (prev2, prev1) = (prev1, Math.Max(prev1, prev2 + x));
    return prev1;
}

// Coin change (minimum coins): dp[a] = min over coins c of dp[a-c] + 1
int CoinChange(int[] coins, int amount)
{
    var dp = new int[amount + 1];
    Array.Fill(dp, amount + 1);                // "infinity" that can't overflow
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        foreach (int c in coins)
            if (c <= a) dp[a] = Math.Min(dp[a], dp[a - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];      // O(amount * coins)
}

// Coin change II (number of ways): loop coins OUTSIDE to count combinations, not permutations
int Change(int amount, int[] coins)
{
    var dp = new int[amount + 1]; dp[0] = 1;
    foreach (int c in coins)
        for (int a = c; a <= amount; a++) dp[a] += dp[a - c];
    return dp[amount];
}

// Longest increasing subsequence: O(n log n) with patience sorting
int LengthOfLIS(int[] nums)
{
    var tails = new List<int>();               // tails[k] = smallest tail of an IS of length k+1
    foreach (int x in nums)
    {
        int i = tails.BinarySearch(x);
        if (i < 0) i = ~i;
        if (i == tails.Count) tails.Add(x); else tails[i] = x;
    }
    return tails.Count;
}
// Word break: dp[i] = can s[0..i) be segmented?
bool WordBreak(string s, IList<string> wordDict)
{
    var words = new HashSet<string>(wordDict);
    var dp = new bool[s.Length + 1]; dp[0] = true;
    for (int i = 1; i <= s.Length; i++)
        for (int j = 0; j < i && !dp[i]; j++)
            dp[i] = dp[j] && words.Contains(s[j..i]);
    return dp[s.Length];
}
```

⚡ Tricky Point

Loop order matters in Coin Change II. Coins outer, amounts inner counts *combinations* ({1,2} and {2,1} counted once). Amounts outer, coins inner counts *permutations* (Combination Sum IV). Same code shape, different answers.

15.4 2-D DP

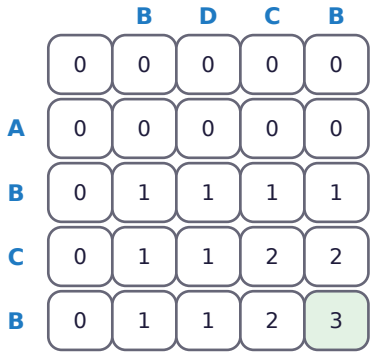
> **✎ Definition**
>
> DP with two indices, usually over two strings or a grid: longest common subsequence, edit distance, unique paths, and the 0/1 knapsack.

Unique paths in a grid

```
int UniquePaths(int m, int n)
{
    var dp = new int[n]; Array.Fill(dp, 1);             // first row all 1s
    for (int r = 1; r < m; r++)
        for (int c = 1; c < n; c++) dp[c] += dp[c - 1];  // from top (old dp[c]) + left
    return dp[n - 1];
}
int MinPathSum(int[][] g)
{
    int m = g.Length, n = g[0].Length;
    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++)
        {
            if (r == 0 && c == 0) continue;
            int up = r > 0 ? g[r - 1][c] : int.MaxValue, left = c > 0 ? g[r][c - 1] : int.MaxValue;
            g[r][c] += Math.Min(up, left);
        }
    return g[m - 1][n - 1];
}
```

Longest Common Subsequence

LCS("ABCB", "BDCB") = 3 ("BCB")



match: dp[i][j] = dp[i-1][j-1] + 1

else: max(dp[i-1][j], dp[i][j-1])

Fill row by row; answer at bottom-right

Figure 15.1 - Filling the LCS table

```
int LCS(string a, string b)
{
    var dp = new int[a.Length + 1, b.Length + 1];        // dp[i,j] = LCS of a[0..i) and b[0..j)
    for (int i = 1; i <= a.Length; i++)
        for (int j = 1; j <= b.Length; j++)
            dp[i, j] = a[i - 1] == b[j - 1]
                ? dp[i - 1, j - 1] + 1
                : Math.Max(dp[i - 1, j], dp[i, j - 1]);
    return dp[a.Length, b.Length];                        // O(n*m)
}
```

Edit distance (Levenshtein)

```
int MinDistance(string a, string b)
{
    int n = a.Length, m = b.Length;
    var dp = new int[n + 1, m + 1];
    for (int i = 0; i <= n; i++) dp[i, 0] = i;            // delete all
    for (int j = 0; j <= m; j++) dp[0, j] = j;            // insert all
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i, j] = a[i - 1] == b[j - 1]
                ? dp[i - 1, j - 1]
                : 1 + Math.Min(dp[i - 1, j - 1], Math.Min(dp[i - 1, j], dp[i, j - 1]));  // replace, delete, insert
    return dp[n, m];
}
```

> **i Note**
>
> Edit distance is useful beyond interviews - for example fuzzy matching of merchant names or narration text during reconciliation.

0/1 Knapsack

Each item can be taken once. `dp[w]` = best value with capacity w.

```
int Knapsack(int[] weight, int[] value, int capacity)
{
    var dp = new int[capacity + 1];
    for (int i = 0; i < weight.Length; i++)
        for (int w = capacity; w >= weight[i]; w--)       // iterate BACKWARDS for 0/1
            dp[w] = Math.Max(dp[w], dp[w - weight[i]] + value[i]);
    return dp[capacity];                                  // O(n * capacity)
}
// Partition equal subset sum = knapsack with bool dp and target = total/2
```

> **⚡ Tricky Point**
>
> In 1-D knapsack, iterating capacity **forwards** lets an item be reused (unbounded knapsack); iterating **backwards** uses each item at most once (0/1 knapsack).

15.5 Other Common DP Families

✎ Definition

Other frequently tested DP families, such as knapsack variants, interval DP, DP on trees, state machines (stock problems) and bitmask DP.

Family	Examples	Typical state
Linear sequences	Climbing stairs, house robber, decode ways	`dp[i]` = answer for prefix i
Grid paths	Unique paths, min path sum, dungeon game	`dp[r][c]`
Two strings	LCS, edit distance, interleaving string, regex matching	`dp[i][j]` over prefixes
Knapsack / subset	0/1 knapsack, partition sum, target sum, coin change	`dp[i][capacity]`
Intervals	Burst balloons, matrix chain, palindrome partitioning	`dp[l][r]`
Palindromes	Longest palindromic subsequence/substring	`dp[l][r]`
State machine	Stock buy/sell with cooldown or fee	`dp[i][holding]`
Trees	House robber III, max path sum	return tuple from children
Bitmask	Travelling salesman, assign tasks	`dp[mask][i]`

```
// Stock with cooldown: states hold / sold / rest
int MaxProfitCooldown(int[] prices)
{
    int hold = int.MinValue, sold = 0, rest = 0;
    foreach (int p in prices)
    {
        int prevSold = sold;
        sold = hold + p;                    // sell today
        hold = Math.Max(hold, rest - p);    // keep holding or buy today
        rest = Math.Max(rest, prevSold);    // cooldown
    }
    return Math.Max(sold, rest);
}

// Longest palindromic subsequence (interval DP)
int LongestPalindromeSubseq(string s)
{
    int n = s.Length; var dp = new int[n, n];
    for (int i = n - 1; i >= 0; i--)
    {
        dp[i, i] = 1;
        for (int j = i + 1; j < n; j++)
            dp[i, j] = s[i] == s[j] ? dp[i + 1, j - 1] + 2 : Math.Max(dp[i + 1, j], dp[i, j - 1]);
    }
    return dp[0, n - 1];
}
```

⚡ Tricky Point

`hold = int.MinValue` then `hold + p` overflows on the first iteration (to a large negative, which happens to be harmless here, but not in general). Safer: initialise `hold = -prices[0]` or use `long`.

> **★ Interview Tip**
>
> DP vs greedy vs backtracking: backtracking explores **all** choices (exponential); DP explores all choices but **reuses** sub-results (polynomial when states are few); greedy commits to **one** choice per step (fastest, but needs a proof).

16. Bit Manipulation and Math

16.1 Bitwise Operators

> ✎ **Definition**
>
> Operators that work on individual bits: AND &, OR |, XOR ^, NOT ~, and the left and right shifts << and >>.

Operator	Meaning	Example (5 = 0101, 3 = 0011)
`&`	AND	5 & 3 = 1 (0001)
`|`	OR	5 | 3 = 7 (0111)
`^`	XOR (1 if bits differ)	5 ^ 3 = 6 (0110)
`~`	NOT	~5 = -6 (two's complement)
`<<`	Left shift (x 2^k)	5 << 1 = 10
`>>`	Arithmetic right shift (/ 2^k, keeps sign)	-8 >> 1 = -4
`>>>`	Logical right shift (C# 11)	-8 >>> 28 = 15

16.2 Essential Tricks

✎ Definition

Common bit tricks: check, set or clear the i-th bit, `n & (n-1)` to clear the lowest set bit, power-of-two checks, XOR to find the single number, and counting set bits.

```
bool IsOdd(int x) => (x & 1) == 1;
bool IsPowerOfTwo(int x) => x > 0 && (x & (x - 1)) == 0;   // power of 2 has a single set bit
int ClearLowestBit(int x) => x & (x - 1);
int LowestBit(int x) => x & -x;
bool GetBit(int x, int i) => ((x >> i) & 1) == 1;
int SetBit(int x, int i) => x | (1 << i);
int ClearBit(int x, int i) => x & ~(1 << i);
int ToggleBit(int x, int i) => x ^ (1 << i);
int CountBits(int x) => System.Numerics.BitOperations.PopCount((uint)x);   // built-in popcount
int SingleNumber(int[] nums)          // every number appears twice except one
{
    int r = 0;
    foreach (int x in nums) r ^= x;   // a ^ a = 0, a ^ 0 = a
    return r;
}

int MissingNumber(int[] nums)         // 0..n with one missing
{
    int r = nums.Length;
    for (int i = 0; i < nums.Length; i++) r ^= i ^ nums[i];
    return r;
}

int[] CountBitsUpTo(int n)            // DP with bits: bits[i] = bits[i >> 1] + (i & 1)
{
    var res = new int[n + 1];
    for (int i = 1; i <= n; i++) res[i] = res[i >> 1] + (i & 1);
    return res;
}
// Enumerate all subsets with a bitmask (n <= 20)
for (int mask = 0; mask < (1 << n); mask++)
{
    var subset = new List<int>();
    for (int i = 0; i < n; i++) if ((mask & (1 << i)) != 0) subset.Add(nums[i]);
}
```

⚡ Tricky Point

Operator precedence: `x & 1 == 1` parses as `x & (1 == 1)` and **doesn't compile** in C# (int & bool). Always parenthesise: `(x & 1) == 1`. Also `1 << 31` is negative for `int`; use `1L << i` for 64-bit masks.

16.3 Math for Coding Interviews

✎ Definition

Maths that appears in coding interviews: GCD and LCM, prime checks and the sieve, modular arithmetic, fast exponentiation, and combinatorics basics.

```
int Gcd(int a, int b) => b == 0 ? a : Gcd(b, a % b);           // Euclid, O(log min)
long Lcm(long a, long b) => a / Gcd((int)a, (int)b) * b;        // divide first to avoid overflow

bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int i = 2; (long)i * i <= n; i++) if (n % i == 0) return false;   // O(sqrt n)
    return true;
}
bool[] Sieve(int n)                                             // primes up to n, O(n log log n)
{
    var composite = new bool[n + 1];
    for (int i = 2; (long)i * i <= n; i++)
        if (!composite[i])
            for (int j = i * i; j <= n; j += i) composite[j] = true;
    return composite;                                            // false (for i >= 2) = prime
}

const int MOD = 1_000_000_007;
long ModPow(long b, long e, long m)                             // (b^e) % m in O(log e)
{
    long r = 1; b %= m;
    while (e > 0) { if ((e & 1) == 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r;
}
int Mod(int a, int m) => ((a % m) + m) % m;                     // C# % can be negative!

int ReverseInteger(int x)
{
    long r = 0;
    while (x != 0) { r = r * 10 + x % 10; x /= 10; }
    return r > int.MaxValue || r < int.MinValue ? 0 : (int)r;    // overflow check
}
```

⚡ Tricky Point

In C#, `-7 % 3` is **-1** (sign follows the dividend), not 2. Use `((a % m) + m) % m` for a mathematical modulo, e.g. circular array indices with negative steps. And `Math.Abs(int.MinValue)` throws `OverflowException`.

Useful formulas

Formula	Value
1 + 2 + ... + n	n(n+1)/2
Number of subsets of n items	2^n
Permutations of n items	n!
Combinations C(n, k)	n! / (k! (n-k)!)
Edges in a complete graph	n(n-1)/2
Nodes in a perfect binary tree of height h	2^(h+1) - 1
Digits of n	floor(log10 n) + 1

17. Patterns, Interview Strategy and Revision

17.1 Problem-to-Pattern Cheat Sheet

✎ Definition

A table that maps problem clues ("sorted array", "top K", "shortest path"...) to the pattern that usually solves them, with its typical complexity.

If the problem says...	Think of	Typical complexity
Sorted array, find pair/triplet	Two pointers	O(n), O(n^2)
Contiguous subarray/substring with a condition	Sliding window	O(n)
Subarray sum equals K (negatives allowed)	Prefix sum + hash map	O(n)
Count, duplicates, "seen before", anagrams	Hash map / hash set	O(n)
Sorted input, "find position", "minimum X such that"	Binary search (on index or answer)	O(log n), O(n log R)
Top K, K-th largest, merge K sorted	Heap / PriorityQueue	O(n log k)
Next greater/smaller, span, histogram	Monotonic stack	O(n)
Sliding window maximum	Monotonic deque	O(n)
Linked list cycle, middle, k-th from end	Fast and slow pointers	O(n)
Tree path, depth, subtree property	DFS recursion (post-order)	O(n)
Tree level by level, shortest steps in grid	BFS	O(V + E)
Connected components, islands, flood fill	DFS/BFS or Union-Find	O(V + E)
Dependencies, ordering, prerequisites	Topological sort	O(V + E)
Weighted shortest path	Dijkstra (non-negative), Bellman-Ford	O(E log V)
All combinations / permutations / subsets	Backtracking	exponential
Min/max ways, "number of ways", optimal choices with overlap	Dynamic programming	polynomial
Intervals, scheduling	Sort + greedy / sweep	O(n log n)
Prefix search, autocomplete, word dictionary	Trie	O(L)
"Appears once", toggles, subsets of small n	Bit manipulation	O(n)
Range sum with updates	Fenwick / segment tree	O(log n)

17.2 Edge Case Checklist

> ✎ **Definition**
>
> The edge cases to test before submitting: empty input, a single element, duplicates, negative numbers, overflow, already sorted or reversed input, and very large inputs.

- Empty input, single element, two elements.
- All elements equal; duplicates.
- Negative numbers, zero, `int.MinValue` / `int.MaxValue` (overflow!).
- Already sorted, reverse sorted.
- Very large n (recursion depth, O(n^2) too slow).
- Strings: empty string, spaces, upper/lower case, Unicode.
- Graphs: disconnected components, self-loops, cycles, unreachable target.
- Trees: null root, skewed (linked-list shaped) tree, single node.
- Off-by-one: inclusive vs exclusive ranges, `<` vs `<=` in loops.

17.3 Common C# Pitfalls in Coding Rounds

> ✎ **Definition**
>
> C#-specific mistakes in coding rounds, such as integer overflow (`int` vs `long`), string concatenation in loops, modifying a collection while iterating, and `PriorityQueue` ordering.

Pitfall	Fix
Integer overflow (`a * b`, `sum`, `lo + hi`)	Use `long`, `lo + (hi - lo) / 2`, `checked` while testing
Modifying a collection inside `foreach`	Iterate a copy or use a `for` loop backwards
`list.Contains` / `Remove` inside loops (hidden O(n))	Use `HashSet` / `Dictionary`
String concatenation in a loop (O(n^2))	`StringBuilder`
Adding `path` instead of a copy in backtracking	`res.Add(new List<int>(path))`
`Dictionary[key]` on missing key throws	`TryGetValue`, `GetValueOrDefault`
Negative modulo	`((a % m) + m) % m`
Deep recursion StackOverflow	Iterative with `Stack<T>` / BFS
Comparator `a - b` overflow	`a.CompareTo(b)`
`PriorityQueue` is a min-heap	Pass a reversed comparer for max-heap
Jagged vs rectangular arrays mixed up	`int[][] g` uses `g[r][c]`; `int[,] g` uses `g[r, c]`

17.4 Must-Practise Problem List (by Topic)

✎ Definition

A curated list of must-solve practice problems, grouped by topic, that cover every pattern in this guide.

Topic	Problems
Arrays & hashing	Two Sum, Contains Duplicate, Valid Anagram, Group Anagrams, Top K Frequent, Product of Array Except Self, Longest Consecutive Sequence
Two pointers	Valid Palindrome, 3Sum, Container With Most Water, Trapping Rain Water
Sliding window	Best Time to Buy/Sell Stock, Longest Substring Without Repeating, Longest Repeating Character Replacement, Minimum Window Substring
Stack	Valid Parentheses, Min Stack, Daily Temperatures, Largest Rectangle in Histogram
Binary search	Binary Search, Search Rotated Array, Find Min in Rotated, Koko Eating Bananas, Median of Two Sorted Arrays
Linked list	Reverse List, Merge Two Lists, Linked List Cycle, Remove Nth From End, LRU Cache, Merge K Lists

Topic	Problems
Trees	Invert Tree, Max Depth, Diameter, Same Tree, Level Order, Validate BST, LCA, Kth Smallest, Serialize/Deserialize
Heap	Kth Largest, K Closest Points, Task Scheduler, Find Median from Data Stream
Backtracking	Subsets, Permutations, Combination Sum, Word Search, N-Queens
Graphs	Number of Islands, Clone Graph, Rotting Oranges, Course Schedule, Pacific Atlantic, Network Delay Time
DP	Climbing Stairs, House Robber, Coin Change, LIS, LCS, Word Break, Edit Distance, Partition Equal Subset Sum
Greedy & intervals	Jump Game, Gas Station, Merge Intervals, Non-overlapping Intervals, Meeting Rooms II
Tries & bits	Implement Trie, Word Search II, Single Number, Counting Bits, Missing Number

17.5 Quick Quiz: Name the Pattern and Complexity

✎ Definition

A self-test: for each short problem statement, name the pattern that solves it and the resulting time and space complexity.

Question	Answer
Find if any two numbers sum to K in an unsorted array	Hash set of complements, O(n) time, O(n) space
Longest substring with at most 2 distinct characters	Sliding window + dictionary of counts, O(n)
Kth smallest element in a BST	In-order traversal, stop at k: O(h + k)
Minimum number of steps from start to end in a maze	BFS on grid, O(rows x cols)
Can all courses be finished?	Topological sort / cycle detection, O(V + E)
Number of ways to decode "226"	1-D DP: dp[i] uses 1- and 2-digit decodes, O(n)
Merge overlapping meeting times	Sort by start + merge, O(n log n)
Find the duplicate number in 1..n without modifying the array, O(1) space	Floyd's cycle detection on index links, O(n)
Largest element in every window of size k	Monotonic deque, O(n)
Detect if a graph is bipartite	BFS/DFS 2-colouring, O(V + E)

17.6 Study Roadmap

✎ Definition

A suggested week-by-week order of study, from complexity and arrays to graphs and dynamic programming, with practice goals for each stage.

Week	Focus	Daily target
1	Chapters 1-4: complexity, arrays, strings, hashing, two pointers, sliding window	3 easy + 1 medium
2	Chapters 5-7: recursion, backtracking, binary search, sorting	2 medium
3	Chapters 8-9: linked lists, stacks, queues, monotonic stack	2 medium
4	Chapters 10-11: trees, BST, heaps	2 medium

Week	Focus	Daily target
5	Chapters 12-14: graphs, union-find, tries, greedy, intervals	2 medium
6	Chapter 15: dynamic programming	2 medium + 1 hard
7	Mixed revision and timed mock interviews (45 min per problem)	2 random problems

> **i Note**
>
> Learn **patterns**, not individual problems. After solving a problem, write one line in your notes: "Pattern: X. Key insight: Y. Complexity: Z." Revisit those notes before interviews - it's far more effective than re-solving everything.

> **★ Interview Tip**
>
> A strong interview answer has four parts: (1) clarify and give an example, (2) brute force with complexity, (3) optimised approach with the key insight, (4) clean code, dry run, edge cases and final complexity.